



An approach to test-driven development of conceptual schemas

Albert Tort*, Antoni Olivé, Maria-Ribera Sancho

Universitat Politècnica de Catalunya – BarcelonaTech, Department of Service and Information System Engineering, Campus Nord, Edif. Omega, Jordi Girona 1–3, 08034 Barcelona (Catalonia), Spain

ARTICLE INFO

Article history:

Received 17 July 2010

Received in revised form 18 July 2011

Accepted 18 July 2011

Available online 29 July 2011

Keywords:

Conceptual modeling

Testing

TDD

Requirements validation

UML/OCL

ABSTRACT

Test-Driven Development (TDD) is an extreme programming development method in which a software system is developed in short iterations. In this paper we present the Test-Driven Conceptual Modeling (TDCM) method, which is an application of TDD for conceptual modeling, and we show how to develop a conceptual schema using it. In TDCM, a system's conceptual schema is incrementally obtained by performing three kinds of tasks: (1) Write a test the system should pass; (2) Change the schema to pass the test; and (3) Refactor the schema to improve its qualities. We also describe an integration approach of TDCM into a broad set of software development methodologies, including the Unified Process development methodology, the MDD-based approaches, the storytest-driven agile methods and the goal and scenario-oriented requirements engineering methods. We deal with schemas written in UML/OCL, but the TDCM method could be adapted to the development of schemas in other languages.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

A conceptual schema defines the general knowledge that an information system needs to know in order to perform its functions [1]. A conceptual schema consists of a structural (sub)schema and a behavioral (sub)schema. The structural schema consists of a taxonomy of entity types, a set of relationship types, and the constraints they must satisfy. The behavioral schema consists of a set of event types (or a similar construct), with their characteristics, constraints and effects. Two fundamental quality properties of a conceptual schema are correctness (i.e. the knowledge defined is true for the domain) and completeness (i.e. all relevant knowledge is defined) [2].

The main objective of the research reported here is the development of a new conceptual modeling approach that fosters the achievement of the above mentioned fundamental qualities. The overall framework of the research is that of design science [3]. The problem we try to solve is the development of correct and complete conceptual schemas. The problem is significant because each information system development project requires the development of its conceptual schema, and that conceptual schema must be correct and complete [4]. On the other hand, it is widely recognized that errors at the conceptual level must be detected and corrected as soon as possible. In this paper we present a new method for the development of conceptual schemas, that we call Test Driven Conceptual Modeling (TDCM), which is a variant of the popular Test-Driven Development (TDD). As far as we know, this is the first work that explores the use of TDD in conceptual modeling. We have evaluated our method by means of a case study consisting of the reverse engineering of a large real-world conceptual schema.

TDD [5,6] is an extreme programming development method [7] in which (1) an exhaustive suite of test programs is maintained; (2) no code goes into production unless it has associated tests; (3) tests are written first; and (4) the tests determine what code needs to be written [8]. In TDD a software system is developed in short iterations. In each iteration, the developer:

* Corresponding author. Tel.: +34 93 413 78 59.

E-mail addresses: albert.tort@upc.edu (A. Tort), antoni.olive@upc.edu (A. Olivé), maria.ribera.sancho@upc.edu (M.-R. Sancho).

(1) Writes a test for the next bit of functionality that he or she wants to add; (2) Writes the functional code until the test passes; and (3) Refactors both new and old codes to make them well structured.

Conceptual schemas written in formalized languages (like UML/OCL) are software artifacts that can be executed to a varying degree [4,9,10]. Then, a question naturally arises: can we test conceptual schemas?

In [11] we have shown that conceptual schemas can be tested. We have identified five kinds of tests that can be applied to conceptual schemas regardless of the conceptual modeling language. These kinds of tests are the assertions of (1) the consistency of the Information Base (IB); (2) the inconsistency of the IB; (3) the occurrence of a domain event; (4) the non-occurrence of a domain event, and (5) the contents of an IB state. We also proposed the Conceptual Schema Testing Language (CSTL), a specialized language for writing automated tests [5] of executable conceptual schemas written in UML/OCL. CSTL makes it possible to write programs to test conceptual schemas in the style of the modern xUnit software testing frameworks [12], which are successfully used in many programming projects, for unit tests and other functional tests. xUnit frameworks and their extensions allow test automation, that is, the use of software to control the execution of tests, the comparison of actual outcomes to predicted outcomes, the setting up of test preconditions, and other test control and test reporting functions [13]. CSTL enables the automation of tests of conceptual schemas when there is a testing environment able to interpret, execute and compute their verdict. We described and implemented a prototype of a conceptual schema testing environment [11]. In this prototype, tests written in CSTL may be automatically executed as many times as needed.

Once we know that conceptual schemas can be tested, the second question that arises is: could we improve the completeness and correctness of conceptual schemas by developing them using the TDD approach?

In this paper we focus on the important and practical issue of determining whether it would be possible to develop a conceptual schema using a TDD-based method, and whether it would be possible to integrate such method with existing software development methodologies. As far as we know, this is the first work that explores the use of TDD in conceptual modeling.

To achieve that objective, we first show that it is possible to develop a conceptual schema using a variant of TDD, which we call Test-Driven Conceptual Modeling (TDCM). Using TDCM, a system's conceptual schema is incrementally obtained by performing three kinds of tasks: (1) Write a test the system should pass; (2) Change the schema to pass the test; and (3) Refactor the schema to improve its qualities. The proposed environment [11] for testing conceptual schemas supports the application of the TDCM method. Then, we explore the integration of TDCM with existing software development methods when they are based on iterative paradigms and they include artifacts to specify the conceptual schema.

The structure of the paper is as follows. In the next section we review the main constructs and notation we have used to define the conceptual schemas that we want to develop. Section 3 briefly reviews the schema testing language we use (CSTL) and the testing environment that makes it possible to test schemas. We refer the interested reader to [11] for details.

Section 4 describes the TDCM method and shows how to develop conceptual schemas using it. We explain the details of the TDCM cycle, and two important guidelines on how to apply the method in some circumstances. We illustrate the method with its application to a fragment of the well-known Meeting Scheduler case study [14]. We also report on the application of TDCM to the full development of a case study consisting of the reverse engineering of a large real-world conceptual schema. In Section 5 we sketch how to integrate TDCM into four popular methods: the Unified Process development methodology [15], the MDD-based approaches [10,16,17], the storytest-driven agile methods [18] and the goal and scenario-oriented requirements engineering methods [14]. In Section 6, we review the related work regarding conceptual schema validation and Test-Driven Development. Section 7 summarizes the conclusions we have reached and points out future research directions.

2. Basic concepts and notation of the conceptual schemas

In this section, we review the main concepts and notation we use to define conceptual schemas. We adopt UML/OCL as the conceptual modeling language, but in principle the ideas presented here could also be applied to schemas written in other conceptual modeling languages [19].

A conceptual schema consists of a structural (sub)schema and a behavioral (sub)schema. The structural schema consists of a taxonomy of entity types (a set of entity types with their generalization/specialization relationships and the taxonomic constraints), a set of relationship types (either attributes or associations), the cardinality constraints of the relationship types and a set of other constraints formally defined in OCL [20]. Entity and relationship types may be base or derived. The population of the base entity and relationship types is explicitly represented in the Information Base (IB). If they are derived, there is a formal derivation rule in OCL that defines their population in terms of the population of other types.

The behavioral schema consists of a set of event types. We adopt the view that an event can be modeled as a special kind of entity, which we call event entity [21]. An event entity is an instance of an event type. In general, there are three kinds of event types: domain events, action requests and queries [1,21], but in this paper, for expository reasons, we will deal only with domain event types. The extension to the other two kinds is straightforward. Given that there is a direct correspondence between domain events and invocations of system operations, the adaptation of our work to languages that view events as invocations of system operations is straightforward [15,22].

Domain event types have characteristics, constraints and effects. The characteristics of an event are the set of relationships (attributes or associations) in which it participates. The constraints are the conditions that events must satisfy in order to occur. An event may occur in the state *S* of the IB if *S* satisfies all constraints and the event satisfies its event constraints. Each event type has an operation called *effect()* that gives the effect of an event occurrence. We adopt the specification style used in the well-known

design-by-contract method [23] and we declaratively define the effect by the postcondition of the operation. We define both the event constraints and the postcondition in OCL.¹

The postcondition defines the state of the IB after the event occurrence. It is assumed that the state of the IB after the event occurrence also satisfies all constraints defined over the IB. Therefore, the effect of a domain event is a state that satisfies the postcondition and all IB constraints. Note that the OCL expressions used in constraints, derivation rules and pre/post conditions are without side-effects. These expressions are evaluated over the IB and their evaluation cannot change the IB.

Given that we want to deal with executable conceptual schemas, we also need a procedural specification of the method of the *effect()* operation. An execution of an *effect()* operation can change the IB. A method is correctly specified if the result it produces always satisfies the postcondition and the IB constraints. UML does not include any particular language for writing methods [24]. In the work reported here, we write the methods of the *effect()* operations using a subset of the CSTL [11].

3. Testing conceptual schemas

Conceptual schemas of information systems can be tested. We adopt here the precise and concise definition of testing proposed by Meyer: "To test a program is to try to make it fail" from which the goal of testing becomes "to uncover faults by triggering failures" [25]. In conceptual modeling, the faults we try to uncover are incomplete or incorrect knowledge.

Testing conceptual schemas require a testing language, in which the conceptual modeler writes programs that test the conceptual schema, and a testing environment in which test programs are executed. In the following, we briefly review the language and the testing environment we use in this paper.

3.1. CSTL

Testing conceptual schemas requires a testing language. For explanation purposes, in this paper we will use the testing language called CSTL (Conceptual Schema Testing Language) [11], but in principle the ideas presented here can also be applied to tests written in other conceptual schema testing languages. CSTL is a language that can be used to write functional tests of conceptual schemas.

A test set of a conceptual schema is a set of one or more CSTL programs. A CSTL program consists of a fixture and a set of one or more test cases. A test case is a set of statements that builds a state of the IB, and executes one or more test kinds applicable to conceptual schemas. Test kinds correspond to assertions about (1) the consistency of the Information Base (IB); (2) the inconsistency of the IB; (3) the occurrence of a domain event; (4) the non-occurrence of a domain event, and (5) the contents of an IB state.

A test case always returns a verdict which may be *Pass*, *Fail* or *Error*. In general, the verdict is *Error* when the conceptual schema or the test case is ill-formed and *Fail* when an assertion specified in the test case is not satisfied [11].

It is assumed that the execution of each test case of a CSTL program starts with an empty IB state. With this assumption, test cases of a program are independent of each other, and therefore the order of their execution is irrelevant. The fixture is a set of statements that create an IB state and define the values of the common program variables. Each execution of a test case starts with the execution of the fixture.

In this paper, IB states are set up by means of the occurrence (execution) of domain events. In order to assert that an event *eventId* (previously created) occurs in the current state of the IB, the conceptual modeler writes the following CSTL statement:

```
assert occurrence eventId;
```

The verdict of this assertion is *Fail* if (1) the event constraints are not satisfied; or, after the execution of the event method, (2) the IB state is inconsistent; or (3) the postconditions are not satisfied. Otherwise, the verdict is *Pass*.

In order to assert that the event *eventId* does not occur in an IB state, the conceptual modeler writes the following sentence:

```
assert non-occurrence eventId;
```

The verdict of this assertion is *Fail* if none of the constraints of the event prevent its occurrence. Otherwise, the verdict is *Pass*. The full details about CSTL constructs, verdict computation rules and test program examples can be found in [11].

3.2. The testing environment

CSTL tests are executed in the context of a conceptual schema testing environment. Fig. 1 shows the main components of a prototype testing environment. This prototype deals with schemas defined in UML/OCL (Section 2) and test programs specified in CSTL (Section 3.1). We have used it while applying TDCM in the example reported here.

The environment for testing conceptual schemas consists of a *test processor* and an *information processor*. The execution of conceptual schemas is done by the *information processor*, while the execution of test programs is done by the *test processor*.

¹ The *effect()* operations, or their equivalent system operations, are the only operations we assume in a schema. Usually event types have no subtypes and therefore those operations need not be redefined in subtypes.

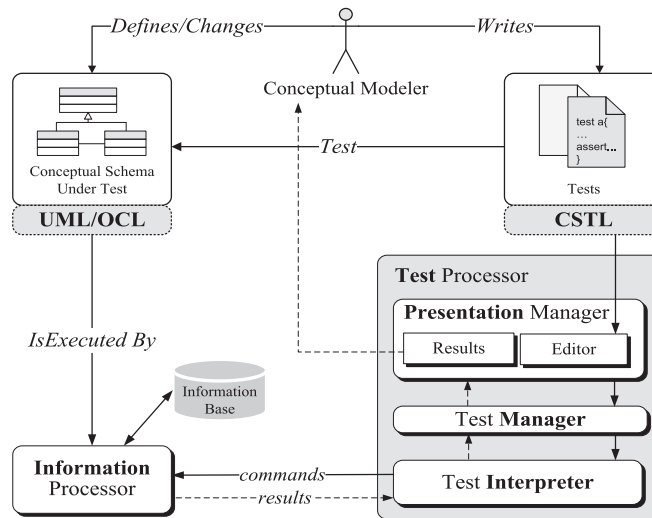


Fig. 1. Conceptual schema testing environment.

The *test processor* reads CSTL programs, executes them and computes their verdicts. When the verdict of a test case is *Error* or *Fail*, the *test processor* provides information about the error or the failure to help the modeler to fix it. It also invokes the services of the *information processor* to initialize or update the IB states involved in test cases.

The description of the prototype and its implementation can be found in [11]. The use of our testing environment applied to the case study of the osCommerce system [26] is detailed in [27].

4. Test-Driven Conceptual Modeling

In this section, we present the main contribution of this paper: the Test-Driven Conceptual Modeling (TDCM) method. TDCM is an iterative method aimed to drive the elicitation and the definition of the conceptual schema of an information system. TDCM uses test cases to drive the conceptual modeling activity. In TDCM, conceptual schemas are incrementally defined and continuously validated.

TDCM is assumed to be used in the context of a requirements engineering (RE) method (Fig. 2). This method determines the source artifacts available when TDCM begins, the iteration sequence, and the condition when TDCM ends. In this section, we define the TDCM method regardless of the RE method in which it is applied, and in Section 5 we provide some details of the integration of TDCM into four well-known RE methods.

A test case written in a conceptual schema testing language is an executable concrete story of a user-system interaction. A test case also specifies user expectations formalized as test assertions. The verdict of a test case is *Pass* if the conceptual schema includes the general knowledge to meet these user expectations.

Test cases that drive the application of TDCM are designed from artifacts provided by the general method. TDCM obtains an executable conceptual schema and a resulting test set that validates the schema. The resulting conceptual schema may be used as a source for the next activities specified by the general method.

4.1. TDCM cycle

In TDCM, a conceptual schema is defined incrementally in short iterations. The iterations that must be performed depend on the RE method in which TDCM is used. The objective of each test case is also given by the RE method in which TDCM is used. An iteration starts by adding a new test case to the passing test set of the previous iteration (*previous test set*).²

The objective of each iteration is to change the schema so that it includes the knowledge to correctly execute the new test case. The previous test set in addition to the new test case is the current test set of the iteration. A TDCM iteration can only finish when the overall verdict of the current test set is *Pass*.

Fig. 3 shows the TDCM cycle and the order of its tasks. A TDCM iteration is a particular instantiation of the TDCM cycle. The TDCM cycle consists of three kinds of tasks: (1) Write a test case; (2) change the schema; and (3) refactor the schema. We describe these tasks in the following.

² We follow here the TDD principle of defining one test case in each iteration, but in practice nothing prevents defining more than one, if so desired.

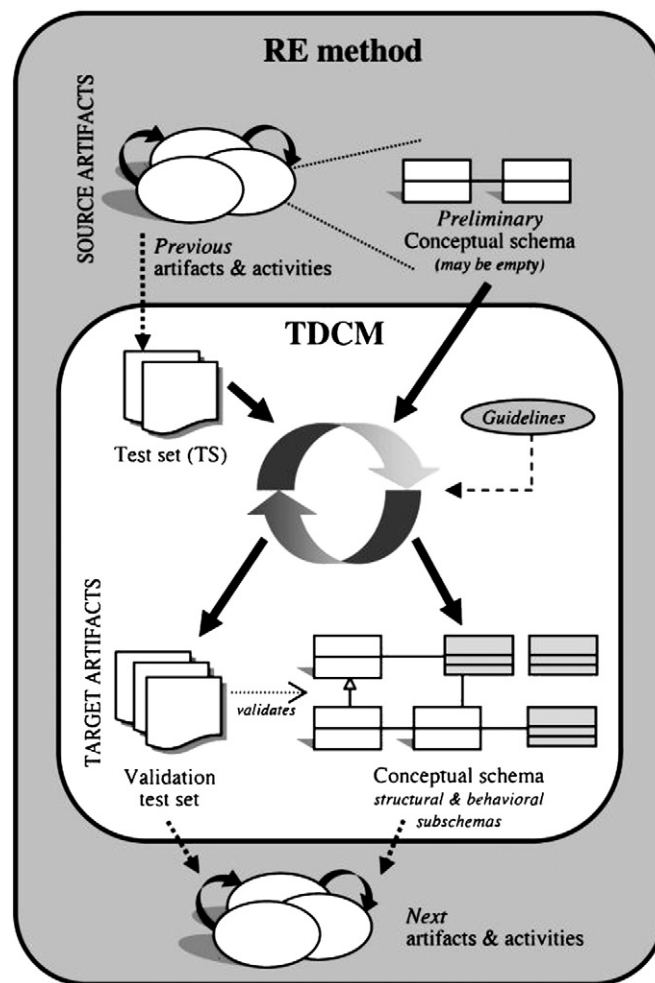


Fig. 2. TDCM context.

4.1.1. Write a test case

The first task of the TDCM cycle consists of setting up a new test case whose verdict will be *Pass* once the conceptual schema includes the knowledge to be added in the current iteration. In the general case, the schema does not include that knowledge initially, and therefore the verdict will be *Error* or *Fail*. At the end of the current iteration, the verdict will be *Pass*.

If the verdict of the *new test case* is *Error* or *Fail*, then the *new test case* is suitable to make progress in the TDCM cycle. If the verdict of the *new test case* is *Pass*, then the iteration objective is already achieved and, consequently, the iteration finishes. Such iterations are sometimes convenient to increase the confidence about already defined knowledge by executing complementary test cases.

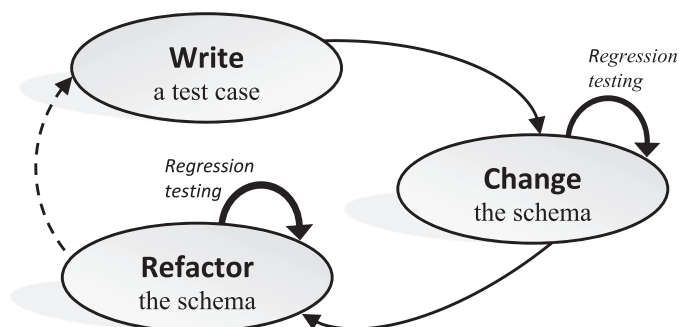


Fig. 3. The TDCM cycle.

Table 1

Failure/error/pass interpretation.

Verdict	Current test case (CTC)	Regression test set (RTS)	
Error	Relevant knowledge needs to be added in the CSUD	Schema element removed	The CSUD becomes incomplete and the deleted schema element needs to be restored
		Relevant constraint added	Modify inconsistent states to maintain consistent test cases
Fail	The knowledge defined in the CSUD needs to be corrected according to the asserted expectations	Both CTC and RTS may Pass without changing them (only changing the CSUD)	The knowledge defined in the CSUD needs to be corrected
		Neither CTC nor RTS can Pass without changing them	Inconsistent requirements
Pass	The CSUD has the necessary knowledge and satisfies the asserted expectations formalized in CTC	The CSUD still has the necessary knowledge and satisfies the asserted expectations formalized in RTS	

4.1.2. Change the schema

Changing the schema to make the verdict of the *new test case* Pass is the focused objective to be achieved by the conceptual modeler in this task. The testing environment provides information about the failure or the error. The next change to be done in the schema is fixing this error or failure.

The changes made in the schema can be validated by the execution of the *new test case* after each change. Changing the schema and checking if the verdict of the *new test case* becomes Pass is the main activity to make progress in TDCM.

If the verdict of the *new test case* is Error, then the required knowledge is not defined in the schema. Information about the error helps to find out required knowledge to be added to the schema. In this way, TDCM drives the completeness of the schema according to the user needs expressed in test cases.

Table 1 summarizes the interpretation of the verdict of the current test set (CTC) and the regression test set (RTS) at each iteration. This interpretation drives the application of TDCM in order to evolve the Conceptual Schema Under Development (CSUD).

If the verdict of the *new test case* is Fail, the schema needs to be changed because it does not produce the expected result (the schema is not correct according to the test assertions). The information about the failure provided by the testing environment helps to find out the changes to be done in the schema in order to fix the failure.

While changing the schema, test cases of the *previous test set* can be automatically executed as regression tests. The purpose of regression testing is to detect if the verdict of any of the previous test cases ceases to be Pass. In this case, a collateral effect on previous defined knowledge of the schema is detected. The problem to be solved is indicated by the failure/error information. Note that test sets written in a conceptual modeling language designed in the xUnit fashion (like CSTL) are automated.

If the verdict of the *previous test set* becomes Fail, then two cases are possible:

- *Previous defined knowledge ceases to be correct due to the last changes:* A derivation rule ceases to derive information as expected, an event occurrence ceases to produce the expected IB state or a domain event ceases to be applicable as expected. The conceptual schema needs to be changed to fix the failure.
- *Inconsistent requirements are revealed:* If it is not possible to change the schema to pass both the *new test case* and the previous failing test cases, an inconsistency between requirements has been revealed. The inconsistency must be resolved. Involved test cases may be changed, if needed, to reflect updated expectations.³

If the verdict of a previous test case becomes Error, then two cases are possible:

- *The conceptual schema has become incomplete:* A schema element that was necessary for a previous test case has been removed.
- *The IB state of a previous test case has become inconsistent:* IB states of previous test cases may become inconsistent when a new constraint is added to the schema. Inconsistent states need to be updated to ensure that test cases formalize consistent stories. In Section 4.4, two guidelines are proposed to minimize this case.

The TDCM method is adaptable to the experience of the conceptual modeler. Smaller changes and more frequent checking of the verdict of the *new test case* provide more failing/error information to guide the definition of the schema. More regression testing provides more confidence about the already included knowledge. It is important to remark that TDCM encourages continuous reflection about the knowledge of the schema and its alignment with user expectations.

When the whole verdict of the *current test set* becomes Pass, the iteration objective is achieved and the conceptual modeler can proceed to the next task.

³ The choice of what to do with the revealed inconsistency depends on the inconsistency handling strategy followed in the project [107]. In this paper, we assume that inconsistencies must be solved when detected, but it would be interesting to explore the use of TDCM with other strategies, specially the one that differs in the resolution and continues the development until such time deemed appropriate to re-visit the inconsistency.

4.1.3. Refactor the schema

Refactoring aims to improve the quality of the conceptual schema without changing the knowledge specified in it. Much research work has been devoted to identify and catalog software anomalies [28]. Refactoring *smells* to detect possible anomalies and refactoring patterns to fix them have also been proposed for UML/OCL schemas (a representative set of papers is [29–33]).

When a conceptual schema is defined in an iterative way, as proposed in TDCM, refactoring may be applied to improve the quality of the schema. TDCM encourages the conceptual modeler to refactor the schema and, in sequence, request the execution of the *current test set*.

If the verdict of the *current test set* becomes *Fail* or *Error*, then we realize that the knowledge of the schema has not been preserved by the refactoring process. The failure/error information provided by the testing environment helps to identify the invalid refactoring changes. If the verdict of the *current test set* is *Pass* and no more refactoring is felt to be needed, then we can start a new iteration.

4.2. Example

We now illustrate TDCM by means of an example based on the Meeting Scheduler system described in [14]. Assume that we have already developed the conceptual schema shown in Fig. 4. Three domain events types are included in this initial schema (two of them are graphically shown in Fig. 4). Their effect is informally described in the following:

- *MeetingRequest*: A new instance of the entity type *Meeting* is created with the indicated subject, the set of possible dates, the user that convenes the meeting (*initiator*) and the users invited to participate in the meeting (*invitedParticipant*).
- *ConstraintsSpecification*: A user specifies his/her excluded dates for a meeting. This information is registered in the IB as an instance of *ConstraintsSet*.
- *UserCreation*: A new *User* is created with his/her *name* and *eMail*.

As an example, we show the formal specification of the postcondition (in OCL) and of the method (in CSTL) of the event *MeetingRequest*:

```
context MeetingRequest::effect ()
  post: (Meeting.allInstances () - Meeting.allInstances ()@pre)
    ->one (m | m.subject=self.subject and m.possibleDate=self.possibleDate and
      m.participant=self.invitedParticipant and
      m.initiator=self.initiator)
method MeetingRequest::effect (){
  self.createdMeeting := new Meeting (subject := self.subject,
    possibleDate := self.possibleDate,
    participant := self.invitedParticipant,
    initiator := self.initiator);
}
```

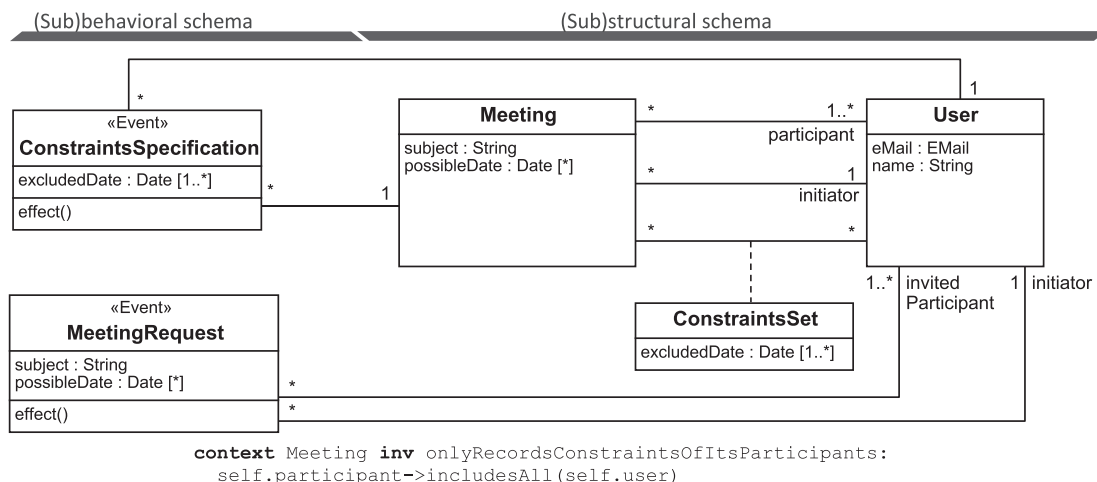


Fig. 4. Initial conceptual schema fragment of the Meeting Scheduler system.

In the following, the application of TDCM is illustrated by means of two example iterations. The purpose of these iterations is to make progress in the elicitation and the definition of the conceptual schema fragment of Fig. 4.

4.2.1. First iteration example: invalid meeting scheduling

Consider the following informal requirement:

“A meeting cannot be scheduled on a conflicting date”

Using TDCM, the first task consists of writing a new test case whose verdict is *Pass* if the conceptual schema satisfies this requirement. In this case, we write the test case *invalidMeetingScheduling*, included in the test program *MeetingScheduling* shown below. Note that the fixture creates two users that will appear in the test case.

```
testprogram MeetingScheduling{
  sarahCreation := new UserCreation(name='Sarah' , eMail:='sarah@sarah.edu' );
  assert occurrence sarahCreation;
  sarah := sarahCreation.createdUser;

  davidCreation := new UserCreation(name='David' , eMail:='david@david.edu' );
  assert occurrence davidCreation;
  david := davidCreation.createdUser;

  test invalidMeetingScheduling{
    mr := new MeetingRequest(subject:='Strategy' ,
      initiator:=sarahCreation.createdUser,
      possibleDate:=Set{ 01-13-2011, 01-15-2011, 01-16-2011} ,
      invitedParticipant:=Set{ sarah, david} );
    assert occurrence mr;
    assert true mr.createdMeeting.participant=Set{ sarah, david} ;

    sarahConstraints := new ConstraintsSpecification(
      meeting:=mr.createdMeeting,
      user:=sarah,
      excludedDate:=Set{ 01-15-2011} );
    assert occurrence sarahConstraints;

    mrs := new MeetingScheduling
      (meeting:=mr.createdMeeting, scheduledDate:=01-15-2011);
    assert non-occurrence mrs;
  }
}
```

The verdict of the *new test case* is *Error* and the test processor informs that this is due to the fact that the *MeetingScheduling* event does not exist in the conceptual schema. In order to fix the error, we add the event type *MeetingScheduling* with its characteristics (Fig. 5, a). For the moment, we need to define neither the effect nor the method.

After this small change in the schema, the verdict of the new test case becomes *Fail* and the test processor informs us that nothing prevents the occurrence of the event *mrs* as expected by the assertion “*assert non-occurrence mrs*”. We add the constraint *MeetingScheduling::doesNotScheduleTheMeetingOnAConflictingDate* in order to fix this failure (Fig. 5, b). The added constraint describes that the scheduled date should not be a conflicting date, but the knowledge about conflicting dates of meetings is not yet in the schema. Therefore, the verdict of the *new test case* is now *Error*. In order to fix it, we need to add the derived attribute *conflictingDate* and its derivation rule (Fig. 5, c).

The derivation rule defines that the set of conflicting dates of a meeting is the union of the set of excluded dates of all the participants of the meeting.

The verdict of the test case *invalidMeetingScheduling* becomes *Pass* and no previous test cases cease to be *Pass*. The iteration objective stated by the *new test case* is now achieved. For the moment, no refactoring is felt to be needed and a new iteration can be initiated.

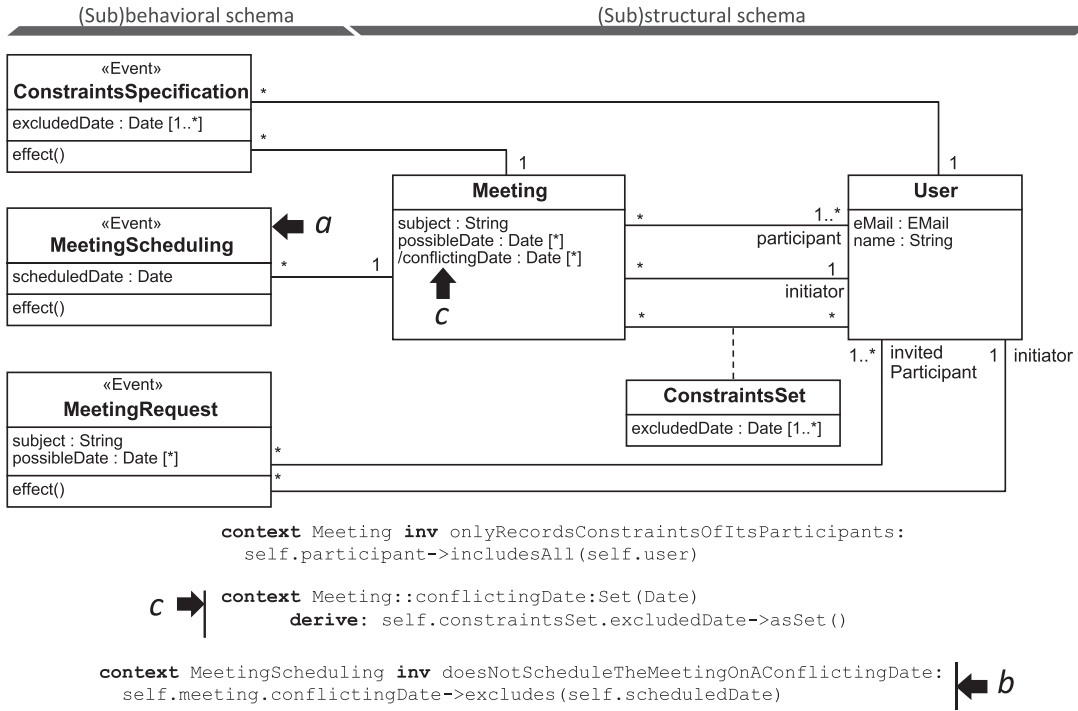


Fig. 5. Schema changes during the first iteration example.

4.2.2. Second iteration example: valid meeting scheduling

In this iteration, we again apply TDCM to evolve the schema of Fig. 5. We add to the previous test program *MeetingScheduling* the following *new test case*, aimed to ensure that a meeting can be scheduled:

```

test validMeetingScheduling{
    mr := new MeetingRequest (subject:='FuturePlans' ,
        initiator:=david,
        possibleDate:=Set{ 02-28-2011, 02-29-2011} ,
        invitedParticipant:=Set{ sarah} );

    assert occurrence mr;

    davidConstraints := new ConstraintsSpecification (
        meeting:=mr.createdMeeting,
        user:=david,
        excludedDate:=Set{ 02-28-2011} );

    assert occurrence davidConstraints;

    mrs := new MeetingScheduling
        (meeting:=mr.createdMeeting, scheduledDate:=02-29-2011);

    assert occurrence mrs;
}
  
```

The verdict of the *new test case* is *Error*. The reason is that the occurrence of the event *davidConstraints* reaches an inconsistent state in which the created meeting violates the integrity constraint *Meeting::onlyRecordsConstraintsOfItsParticipants*.

The conceptual modeler realizes that user *david* is not a participant as it is expected by the story formalized in the *new test case*. We have just detected an inconsistency between two requirements. Therefore, we need to decide whether the initiator of a meeting is a participant by default (as expected in the *new test case*) or not (as expected in the previous test case).

We assume the domain rule that the initiator of a meeting is one of its participants, and therefore we change the effect and the method of the domain event *MeetingRequest* to reflect this:

```
context MeetingRequest::effect()
post: (Meeting.allInstances() - Meeting.allInstances()@pre)
      ->one(m | m.subject=self.subject and m.possibleDate=self.possibleDate and
           m.participant=self.invitedParticipant->including(self.initiator) and
           m.initiator=self.initiator)

method MeetingRequest::effect(){
  self.createdMeeting := new Meeting(subject := self.subject,
                                     possibleDate := self.possibleDate,
                                     participant := self.invitedParticipant
                                     ->including(self.initiator),
                                     initiator := self.initiator);
}
```

The verdict of the *new test case* is now *Error*, because the method and the postcondition of the *MeetingScheduling* event are not specified and consequently, it is not possible to assert its occurrence. At this point, TDCM drives the conceptual modeler to define the effect postcondition and the method of the *MeetingScheduling* event:

```
context MeetingScheduling::effect()
post: self.meeting.scheduledDate=self.scheduledDate

method MeetingScheduling::effect(){
  self.meeting.scheduledDate := self.scheduledDate;
}
```

The verdict of the new test case continues to be *Error* because the entity type *Meeting* does not have the attribute *scheduledDate*. After adding this attribute, the verdict of the new test case is *Pass*. However, the verdict of the previous test set becomes *Error* unless the multiplicity of the attribute *scheduledDate* is set to 0..1 (there must be meetings without an scheduled date). Note that TDCM also drives the consistency about the behavioral and the structural part of the schema. After the addition of the *scheduledDate* attribute and its multiplicity (Fig. 6, a), the verdict of the new test case becomes *Pass* and no collateral effects in previous added knowledge are detected by the automated execution of the *previous test set* (regression testing).

The last task of the iteration is refactoring (if it is applicable). [33] presents the refactoring smell “rule exposure” for OCL expressions (a domain rule is specified in the postcondition or the constraint of an event).

The modified postcondition of the event *MeetingRequest* shown above includes the static domain rule “the initiator of a meeting is always one of its participants”. This domain rule should be represented in the structural part of the schema and removed from the postcondition. A possible refactoring consists in making the relationship type *participant* derived (Fig. 6, b). Its derivation rule specifies that the participants of a meeting are the explicitly invited participants (*invitedParticipant*) and the *initiator* of the meeting. The verdict of the *current test set* is *Pass* after refactoring and consequently, no changes in the conceptual schema knowledge are detected. The iteration finishes.

4.3. The osTicket case study

Ticket support systems allow customers to register and keep track of support requests, and allow staff members to organize, manage and respond to them. osTicket [34] is a widely-used support ticket system available as free and open source software.

We have applied TDCM to the development of the conceptual schema of the osTicket system. The full details of this experiment are reported in [35].

Chikofsky and Cross [36] state that “Reverse engineering is the process of analyzing a subject system to identify the system's components and their interrelationships and create representations of the system in another form or at a higher level of abstraction. The purpose of reverse engineering is to understand a software system in order to facilitate enhancement, correction, documentation and redesign”. In this case study, TDCM is applied to perform reverse engineering in order to obtain the conceptual schema of an existing system. Moreover, the case study is aimed to analyze the application of TDCM to a real-world case. This case study has also been useful in determining and refining the method guidelines and improving our testing processor prototype. This case study is based on the principles of design science research, which state that “knowledge and understanding of a problem domain and its solution are achieved in the building and application of the designed artifact” [3].

Table 2 summarizes general and aggregated data about the case study. The development started with an empty conceptual schema and two sets of informally defined requirements: 1) a set of expected valid configurations of the system and 2) a set of user stories about the creation, management and tracking of support tickets. These stories have been elicited by using the system and

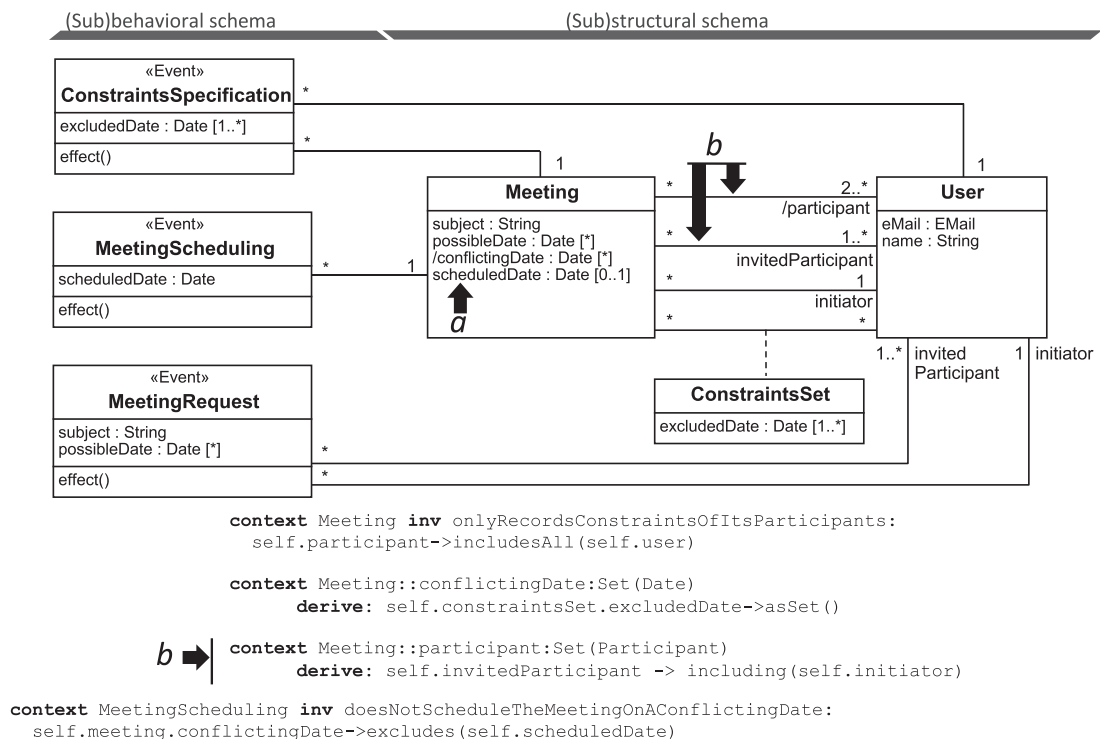


Fig. 6. Schema changes during the second iteration example.

its documentation. The set of stories were ordered following a previously defined testing strategy, based on the complexity of the stories and their dependencies.

In each iteration, a story was formalized as a CSTL test case. The conceptual schema was incrementally developed by fixing the errors and failures information provided by the CSTL processor. The obtained result is an executable conceptual schema and a test set that validates it.

Before starting the experiment, we adapted the CSTL processor in order to automatically collect information about the TDCM application. We obtained information such as the errors and failures provided by the CSTL processor, the time spent to complete each iteration, the schema elements covered by each test case, and the evolution of the conceptual schema under development.

The objectives of this case study were: (1) Exemplifying that it is possible to develop a conceptual schema by using TDCM; (2) Categorizing the most common kinds of errors and failures obtained when applying TDCM and the actions which are performed to fix them; (3) Pointing out initial observations about the common properties of TDCM iterations, and (4) Observing to what extent the main TDD features are applicable to conceptual modeling.

Fig. 7 shows the time spent in applying TDCM in each iteration. One observation is that the time spent in evolving the conceptual schema (by fixing errors/failures) was greater than the time spent in writing the test cases. Moreover, most of the test cases were productive because they led to a change in the schema. The exceptions are those iterations whose verdict was *Pass* in the first execution (they increased the confidence about the validity of the schema, but they did not drive changes).

Table 2

Aggregated data about the osTicket case study.

TDCM iterations			
<i>osTicket conceptual schema</i>			
Classes	28	Number of iterations	100 iterations
Attributes	92	Total development time of the iterations	20 h 11 min
Associations	44	Total time to specify test cases	6 h 23 min
Event types	24	Total time to evolve the conceptual schema under development	13 h 48 min
Integrity constraints	51	Average of lines per test case	2002 lines
<i>osTicket Test Set</i>			
Number of test cases	101	Average of testing specification time per iteration	7,6 minutes
Lines of test cases	2002	Average of conceptual schema development time per iteration	16,4 minutes
Fixture components	25	Average of changes to the conceptual schema per iteration	4 changes

We can see that there are peaks of effort in the iterations in which previously used testing structures are not reused. In general, the testing effort also decreases as we become more familiar with the testing language and the domain, and the schema approaches its completion.

The most common errors and failures revealed in this case study correspond to missing relevant types, invalid definitions of domain event types and failing assertions (either due to incorrect domain event effects, invalid static constraints or incorrect derivation rules). We identified and categorized the errors and failures and their associated fixing actions.

Table 3 summarizes the categorization of errors and failures which drove the conceptual schema development of the osTicket system. This categorization may help in the application of TDCM. The time spent on fixing errors/failures is not always proportional to the number of errors fixed in each iteration. The analysis suggests that missing relevant types are trivial to be fixed (they need to be added). However, the changes to fix failing assertions about the state of the domain or incorrect domain event occurrences may require different actions such as changing derivation rules, integrity constraints or the precondition and postcondition of the effect of the event.

In Fig. 8 we can observe that, in first iterations, the main errors found correspond to relevant types which were not in the schema. These iterations process test cases about the basics and configuration of the system. The action to solve this kind of errors is trivial: to add them to the conceptual schema. However, once the main structural schema elements were specified, the type of failures and errors that drove the schema evolution changed significantly. As we added the first domain events, we detected inconsistent states that required refining static constraints and correctly specifying the effect of these events, but only a small amount of structural knowledge was required to be added or refined.

Before concluding this section we think that it is important to compare the obtained conceptual schema with the one that could presumably be obtained by using a (traditional) method that does not test schemas. We take as the basis for the comparison the two fundamental quality properties of conceptual schemas: completeness and correctness. On this basis, we can conclude that, by using TDCM, the conceptual modeler can be confident that the obtained schemas are complete and correct with respect to the defined test cases. In general, it is not possible to achieve such confidence when schemas are developed using a method that does not test them.

4.4. Guidelines

The iterative nature of TDCM implies that new knowledge is added to the conceptual schema incrementally. The added knowledge may change the set of valid IB states considered in previous iterations. In this section, we present two guidelines aimed at avoiding the need to fix previous test cases due to this reason.

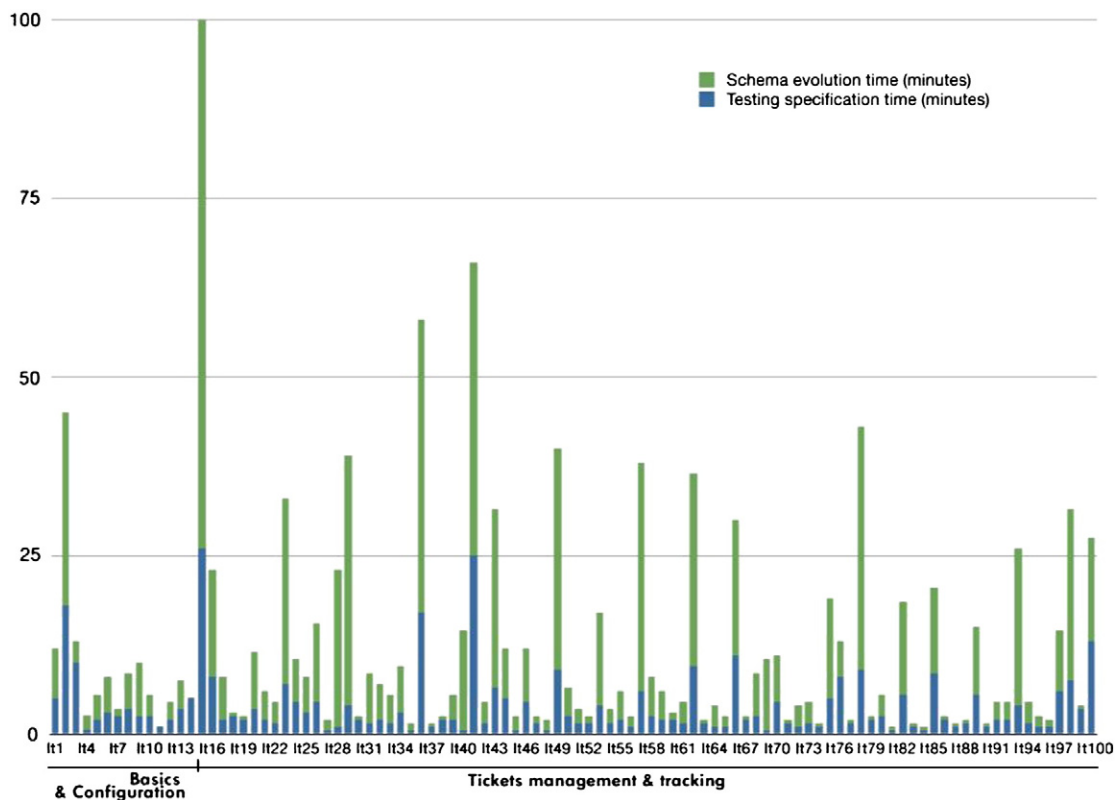


Fig. 7. Time invested in each iteration by applying TDCM.

Table 3

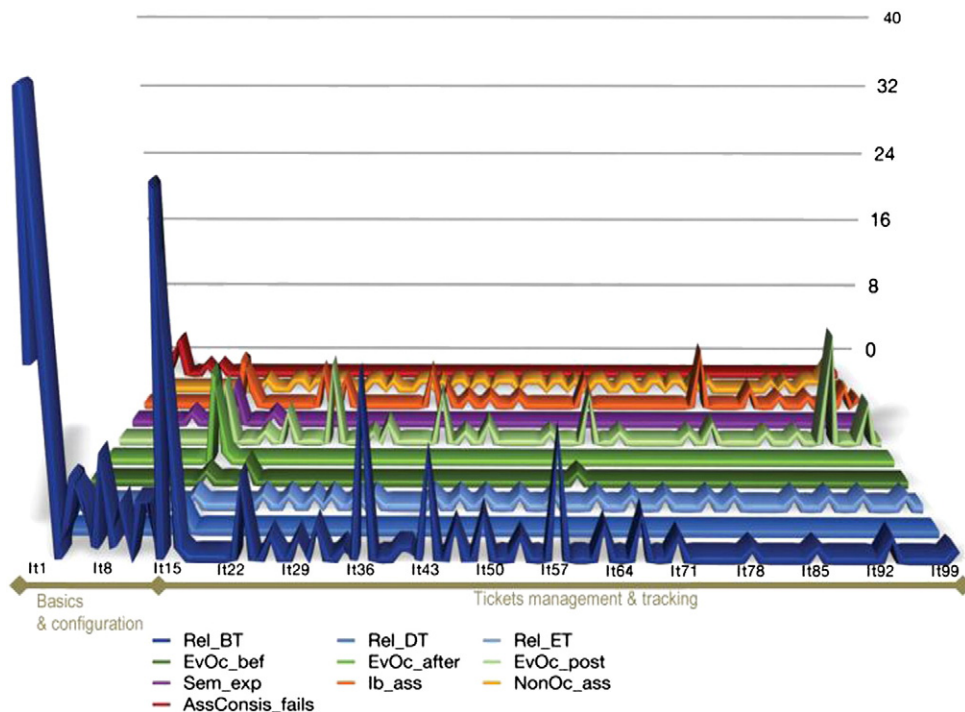
Errors and failures revealed during TDCM application in the case study.

Code	Description	Suggested changes to the schema to fix the error/failure	Number of times revealed	Percentage (%)
Rel_BT	An expected relevant base type (entity type or attribute or association) is not specified in the conceptual schema.	Specify the base type in the conceptual schema.	210	52,4
Rel_DT	An expected relevant derived type is not specified in the conceptual schema.	Specify the derived type (and its derivation rule) in the conceptual schema.	2	0,5
Rel_ET	An expected relevant event type (domain event or query) is not specified in the conceptual schema.	Specify the event type (and its effect or answer) in the conceptual schema.	21	5,2
EvOc_bef	The IB state before an expected event occurrence is inconsistent (the event specification is invalid).	Some (too restrictive) static constraints or preconditions are updated.	4	1,0
EvOc_after	The IB state after an expected occurrence of an event is inconsistent (the event specification is invalid).	The event postcondition, the event method or a static constraint is updated.	10	2,5
EvOc_post	The postcondition is not satisfied after an expected event occurrence.	Either the method or the postcondition are updated.	56	14,0
Sem_exp	An OCL expression in a test case or in the conceptual schema are not valid or inconsistent (e.g. invalid operations for specific types).	Either the expression in the test case is corrected or an element of the schema needs to be changed according to the semantic error revealed.	7	1,7
Ib_ass	A test assertion about the IB state fails.	The effect of an event type or a derivation rule needs to be corrected.	39	9,7
NonOc_ass	A test assertion about the non-occurrence of an event fails.	An event initial integrity constraint (postcondition) needs to be added/updated.	44	11,0
AssConsis_fails	A test assertion about the consistency of an IB state fails.	A static constraint prevents the IB state to be consistent and it is updated.	8	2,0

4.4.1. Guideline: Define constraints as soon as they are noticed

When adding a constraint in the schema, the number of consistent IB states decreases. Therefore, IB states in some previous test cases may become inconsistent and those test cases need to be updated in order to preserve consistent stories. This issue can be minimized if constraints are added to the schema as soon as they are noticed.

Every time that in a new test case we define a new entity or relationship type, we should also define all potential constraints involving that type, and satisfied by the new test case, even if we are not sure that future test cases will also satisfy them. If, later on, the need arises to remove one constraint, this can be done easily because in most cases the removal will not affect the test cases of the previous test set.

**Fig. 8.** Kinds of errors/failures revealed per iteration.

In the initial schema of the example (Fig. 4) we state the constraint that each meeting has at all times at least one participant (multiplicity 1..*). All test cases written since then must specify at least one participant for each meeting. If, later on, we learn that sometimes we do not know yet the participants of a meeting, then we can weaken that constraint (multiplicity *) without impacting previous test cases. In contrast, if the initial multiplicity had been * and later on we need to change it to 1..*, it might be necessary to change previous test cases.

4.4.2. Guideline: Use default values when adding properties to an existing type

New properties (attributes or associations) may need to be added to an entity or domain event type in a new iteration. If the added property is mandatory, previous test cases that use the type of schema which owns this property will become erroneous (because the property was not specified when the previous test cases were written).

If the added property was not considered in previous test cases it is probably because it was not relevant for their testing objectives (the value of the property does not affect the assertions of the test case). Changing previous tests due to this reason can be avoided by specifying a default value for the added property.

Consider, as an example, that we start a new TDCM iteration by adding the following test case to the test program *MeetingScheduling*.

```
test assistantInitiatesAMeeting{
    biancaCreation := new UserCreation(name='Bianca' ,
                                      eMail='bianca@bianca.edu' );

    assert occurrence biancaCreation;
    bianca := biancaCreation.createdUser;

    mr := new MeetingRequest(subject='Interview' ,
                           initiator:=david,
                           urgent:=true,
                           possibleDate:=Set{ 03-15-2011, 04-16-2011} ,
                           participant:=Set{ david, sarah, bianca} );

    assert occurrence mr;

    sarahConstraints := new ConstraintsSpecification(
        meeting:=mr.createdMeeting,
        user:=sarah,
        excludedDate:=Set{ 03-15-2011} );

    assert occurrence sarahConstraints;

    mrs := new MeetingScheduling
        (meeting:=mr.createdMeeting, scheduledDate:=03-15-2011);

    assert occurrence mrs;
}
```

This test case specifies a story in which a meeting is expected to be scheduled even if the meeting is considered to be urgent. The test case specifies that the user *david* requests a meeting and indicates that the users *david*, *sarah* and *bianca* are expected to be participants of the meeting. Although the user *sarah* specifies a date constraint that conflicts with the date in which *david* schedules the meeting, it is expected that the meeting will be successfully scheduled due to its urgency.

The verdict of the new test case is *Error* because the knowledge about the urgency of meetings is not in the schema. We need to add the attribute *MeetingRequest::urgent:Boolean* [1]. We also need to add the attribute *Meeting::urgent:Boolean* [1] and change the postcondition of the event *MeetingRequest* to indicate that, when the meeting is created, the value of this attribute is set according to the value of *MeetingRequest::urgent*.

After these changes, the verdict becomes *Fail* because the event *mrs::MeetingRequestScheduling* cannot occur (as expected) due to the existing initial integrity constraint *MeetingScheduling::doesNotScheduleTheMeetingOnAConflictingDate*. We change this constraint in order to allow urgent meetings to be scheduled on conflicting dates:

```
context MeetingScheduling inv doesNotScheduleTheMeetingOnAConflictingDate:
    not(self.meeting.urgent) implies self.meeting.conflictingDate->excludes(self.scheduledDate)
```

The verdict of the current test case becomes *Pass*. Nevertheless, the verdicts of the previous test cases (see Section 4.2) are *Error*. The reason is that the attribute *MeetingRequest::urgent:Boolean* [1] is mandatory and it is not specified in previous test cases when meetings are requested. In order to achieve the verdict *Pass* for the test set, we could manually update all previous test cases to specify a value for the new mandatory attribute. However, according to the proposed guideline, this task can be avoided by setting its default value to *false*.

5. Integrating TDCM into existing software methods

In Section 4, we defined the TDCM method regardless of the general method in which it is used. The proposed method is applicable to different kinds of projects and may be integrated into existing software development methods when they are based on iterative paradigms and they include artifacts to specify conceptual schemas.

TDCM may be used even if the conceptual schema to be developed is the main purpose of the project. For example, TDCM can be applied when the developed conceptual schema is going to be used as a reference model, when it is a metaschema to be instantiated in several applications or in conceptual modeling education to assist students and novel designers to develop conceptual schemas.

In this section, we present an integration approach of TDCM into the Unified Process [15], into MDD approaches [10,16,17], into goal and scenario-oriented methods in Requirement Engineering (RE) [14,37], and into hybrid methods [38] that combine agile storytest-driven development [18] with conceptual modeling. For each method, we determine the artifacts that may be used to specify the conceptual schema (*target artifacts*) and we describe an integration context to develop them by applying TDCM. We also suggest the artifacts that may be used as a base for the TDCM application (*source artifacts*). Source artifacts may contribute to suggest test cases for the TDCM application. These artifacts can also contribute to prioritizing and planning the processing order of test cases. Finally, we describe the *finalization condition* of the TDCM iteration sequence for each integration case.

Table 4 summarizes the integration points for each method discussed in the following.

5.1. Unified process

The Unified Process (UP) [15] is an iterative method for software development. UP organizes the development process into four phases (inception, elaboration, construction and transition). In each phase, artifacts are started or refined by applying a practice. UP proposes several practices and guidelines to develop each artifact. Each UP project is a particular instantiation of the process that selects a subset of the proposed artifacts and the convenient practices used to develop them.

UP encourages including new practices from other iterative methods. Larman [15] explains that “the set of possible artifacts described in the UP should be viewed like a set of medicines in a pharmacy. On a UP project, a team should select a small subset of artifacts that addresses its particular problems and needs”.

In the UP, the conceptual schema is specified in several artifacts (target artifacts). The structural schema is defined in a UML class diagram in the design model. System events are specified as operations and their effects are defined in a contract with pre and postconditions which can be formally expressed in OCL. By applying TDCM, we can obtain, in a systematic way, the conceptual schema specified in the structural design model and the formal specification of the events (and its procedural methods). We also obtain a test set that validates the schema.

The application of TDCM in a UP project should be performed at the elaboration phase (Fig. 9). UP proposes guidelines and techniques to develop the conceptual schema. UP suggests reusing reference schemas for many common domains, managing category

Table 4
TDCM integration summary.

Integration method		Source artifacts	TDCM iteration sequence	TDCM finalization conditions	Target artifacts
Unified Process (UP) Larman, C. [15]		• Use case model • System Sequence Diagrams (SSDs) • Domain model (may be empty)	Determined from the order in which use cases are iteratively defined.	For each use case scenario, there is at least a representative concrete scenario formalized as a test case.	• Structural design model • System events specification (contract-based) • Validation test set
Model-Driven Development (MDD) MDA [10,16,17]		• Use case model • Scenario models (specified as sequence diagrams) • Preliminary conceptual schema (may be empty)	Determined from the defined use cases and scenarios.	For each use case scenario, there is at least a representative concrete scenario formalized as a test case.	• Structural schema (PIM) • Behavioral schema (PIM) • Validation test set
Storytest-driven agile methods Storytest-Driven Development [18]		• User stories informally defined • Structural schema sketch (may be empty)	Determined from the order in which user stories are iteratively defined.	For each user story, there is at least a test case that formalizes the story.	• Structural schema • Behavioral schema • Validation test set
Goal and scenario-oriented methods GORE [14]	Specification of operations	• List of operations	Determined from the list of operations which are expected to operationalize the goals.	For each operation signature, there is at least a test case that tests it in a consistent state.	• Operational model: operations specification to operationalize the goals. • Object model • Validation test set
	Object and operation model refinement and validation according to scenarios	• Operation model • Behavioral model (scenarios)	Determined from the defined scenarios.	For each defined scenario, there is at least a representative concrete scenario formalized as a test case.	• Operation model • Behavioral model • Object model • Validation test set

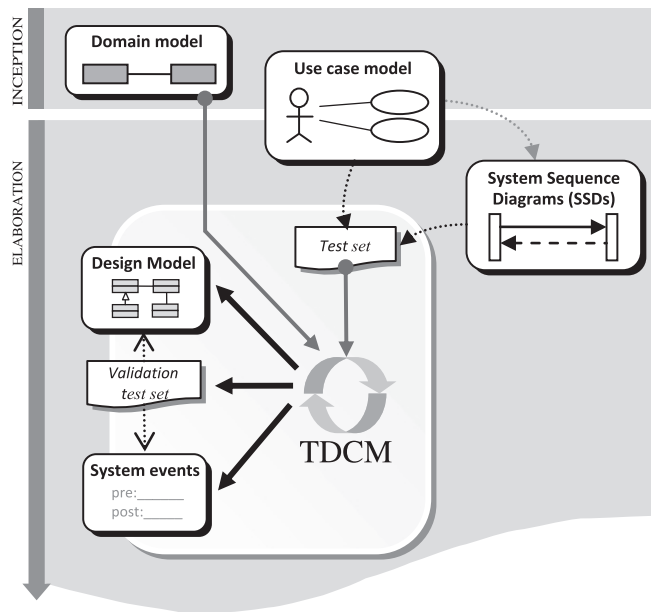


Fig. 9. TDCM integration into the Unified Process.

lists or analyzing use case sentences to find concepts and relationships between them. We propose TDCM as a UP practice aimed at developing the structural and the behavioral parts of the conceptual schema of an information system under development.

The source artifacts for TDCM application are:

- Domain model: It is a preliminary sketch with some static entity and relationship types used in the inception phase. This model, if not empty, can be used as the initial schema to be evolved by applying TDCM.
- Use case model: It specifies the expected functionalities of the system. It is composed of the *use case diagram* and textual specifications of the representative system-actors interactions for each use case.
- System Sequence Diagrams (SSD): UP considers that a use case is often too complex to be defined or processed in a short iteration. Therefore, SSDs are used to specify scenarios for each use case. A test case to validate each SSD diagram can be written to drive the definition of the conceptual schema by applying TDCM.

The TDCM iteration sequence would be determined by the use cases and their corresponding SSDs. For each representative use case scenario, there should be at least one test case to be processed in TDCM iterations. The iteration sequence ends once all use case scenarios have been considered.

Additionally, we obtain the event methods (a procedural specification of their effects). Event methods are of particular interest when a UP project uses UML/OCL conceptual schemas as a blueprint (relatively detailed model to allow code generation) or as a programming language (a complete executable specification in a conceptual schema centric development context [4]).

5.2. MDD approaches

MDD [10,16,17] is a software development paradigm in which a system is developed as a set of transformations from Platform-Independent Models (PIM) to Platform-Specific Models (PSM).

The Object Management Group (OMG) adopted the Model-Driven Architecture (MDA) [16,17] to support MDD. MDA prescribes certain kinds of models to be used in MDD and specifies relationships between these models.

In MDD approaches, conceptual schemas are specified as PIMs. [10] suggests defining the structural schema in a UML class diagram and the behavioral schema as state machines with procedures that specify events as a sequence of actions. TDCM can be used to incrementally define and evolve these models (target artifacts) (Fig. 10).

In MDD, the UML class diagram is obtained as a manual abstraction process from the domain, assisted by guidelines. MDD promotes the use of executable conceptual schemas in order to validate and verify them. In particular, [10] proposes to check class diagrams with expected instantiations. In this context, TDCM is a systematic method that puts this idea into practice while defining the schema.

Before modeling the behavioral schema in UML artifacts, it is suggested that use cases are described and scenarios represented as sequence diagrams.

The source artifacts for applying TDCM are:

- Preliminary structural schema: It is a PIM with static knowledge manually derived from previous artifacts such as the use case model. This model, if not empty, can be used as the initial schema to be evolved by applying TDCM.

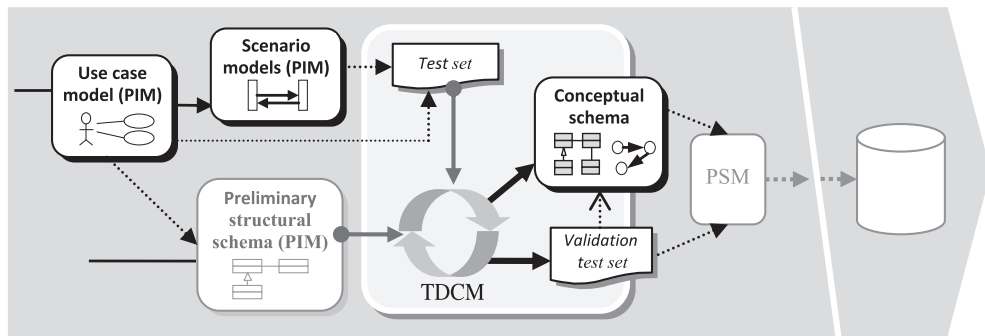


Fig. 10. TDCM integration into an MDD approach.

- Use case model: It specifies the expected functionalities of the system. It is composed of the *use case diagram* and textual specifications of the representative system-actors interactions for each use case.
- Scenario models: They are PIMs that specify representative use case scenarios as sequence diagrams.

The TDCM iteration sequence would be determined by the use cases and their corresponding scenario models. For each representative use case scenario, there should be at least one test case to be processed in TDCM iteration. The iteration sequence ends once all use case scenarios have been considered.

After the application of TDCM, we obtain a PIM that models the conceptual schema (comprising both the structural and behavioral aspects) of the system-to-be. The resultant conceptual schema is correct and complete according to the set of concrete scenarios specified as test cases. The resulting test set validates these quality properties.

Moreover, we obtain a set of methods that specify in a procedural way the effect of the events which make the conceptual schema executable. Methods can be used to facilitate the transformation from PIMs to PSMs, which can be manually performed or automated by model compilers. In contexts in which the conceptual schema is sufficient to obtain executable components of the system [4,9,39], no transformation to PSMs needs to be performed, because the conceptual schema is executable software.

5.3. Storytest-driven agile methods

Storytest-Driven Development [18] is an agile method based on eXtreme Programming (XP) principles [7]. Agile development methods were originally conceived in contrast to plan-driven development. In Storytest-Driven Development, the dominant form of communicating the requirements is user stories. User stories are domain-oriented descriptions of concrete use examples of the system [18]. Stories are formalized as program tests which are intended to be executed in the implementation. [40] argues that in Storytest-Driven Development, tests should be used as a shared language that forces the transformation of ambiguous requirements into executable tests. He also summarizes that storytest-driven development is about “specifying by example”.

Some authors argue that agile methods are a reaction to “documentation driven, heavyweight software development process” [3]. Nevertheless, many authors advocate that plan-driven development and agile development can and should be used in conjunction. For instance, [38] indicates that “hybrid approaches that combine both methods are feasible and necessary. [25] states that program tests “even a million of them, are instances; they miss the abstraction that only a specification can provide”, so that “tests are not substitute for specifications”. In summary, these authors claim that the main aim of agile development is “efficiently respond to changes”, and models can contribute to agility by providing a base to efficiently respond to changes during software development. Class diagram sketches are promoted to “think before you act”.

In projects in which requirements elicitation is performed by capturing stories and the conceptual schema is explicitly modeled in one or more artifacts (target artifacts), then TDCM can be used to define the schema (Fig. 11). Stories can be formalized as conceptual schema tests, as it can be observed in Section 4.2. These stories can be executed at the conceptual modeling stage, providing traceability between the captured stories and the test cases that guide the conceptual modeling activity.

In this context, the source artifacts for applying TDCM are the set of user stories and the conceptual schema sketch (if it exists).

The target artifacts are the diagrams that specify the conceptual schema in an executable form and at the desired level of detail.

User stories are elicited and informally specified iteratively. The TDCM iteration sequence would be determined by these stories. For each story, there should be at least one iteration (test case) that formalizes it. The iteration sequence ends once all stories have been formalized as test cases and all test cases have been processed.

By applying TDCM, contradictions between stories (i.e. requirement conflicts) can be detected (the verdict of more than one non-passing test cases cannot become *Pass* unless at least one test case is changed). Using the terminology of [41], we can say that there is a negative interaction between the functional requirements captured by the stories, because the satisfaction of one of them is reduced as the result of satisfying the other one.

Moreover, TDCM can reveal the necessity of describing new stories. If the set of defined stories is prioritized, then TDCM may process user stories according to this prioritization.

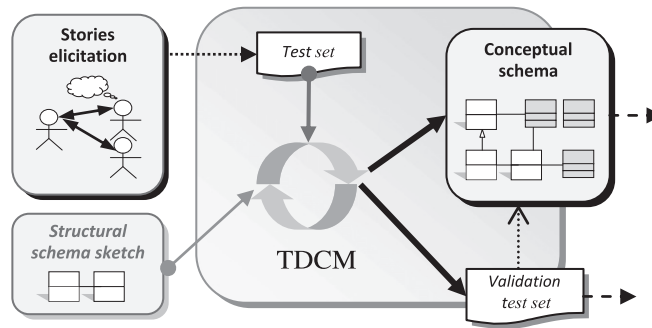


Fig. 11. TDCM integration into Story-Test Driven Development.

5.4. Goal and scenario-oriented methods

Goal-Oriented Requirements Engineering (GORE) advocates the use of goals for requirement elicitation, evaluation, definition and validation. A goal is a prescriptive statement of intent that the system should satisfy through the cooperation of its agents (active system components playing a specific role in goal satisfaction) [14].

Several approaches to identify goals and refine them have been proposed. Goal refinement can be modeled in diagrams (Fig. 12). Leaf nodes are goals whose responsibility can be assigned to a single agent. Leaf goals that can be assigned to a single agent are requirements.

In the following we propose the integration context for using TDCM in the goal-oriented requirements engineering approach which has been recently proposed by Van Lamsweerde [14]. This approach is based on the development of several interrelated models:

- Goal model: It specifies the system's goals in terms of individual features, such as their specification, elicitation source or priority, potential conflicts, etc.
- Obstacle model: It specifies conditions (in the form of a special kind of goals) that may obstruct the realization of the goals defined in the goal model. It is a goal-form of a risk analysis.
- Object model: It models the structural view of the system by means of the static knowledge that concerns the realization of goals.
- Agent model: It models the responsibility view of the system. It specifies the agents that are responsible for each goal.
- Operation model: It models the functional view of the system. It specifies the operations (by means of pre and postconditions) that operationalize each goal.
- Behavior model: It models the behavioral view of the system in terms of scenarios for capturing interaction (operations execution) among agents. In requirements engineering, goals and scenarios complement each other [14,42,43]. Heuristics are suggested to ensure that the set of scenarios cover the goals, by considering both positive and negative scenarios.

In this approach, the conceptual schema is defined by the object model (structural conceptual schema) and the operation model (behavioral conceptual schema).

TDCM may be applied for two different purposes (Fig. 13) in the context of this GORE approach: 1) Specify the operations that operationalize the goal model, and 2) refine and validate the object and the operation model according to scenarios.

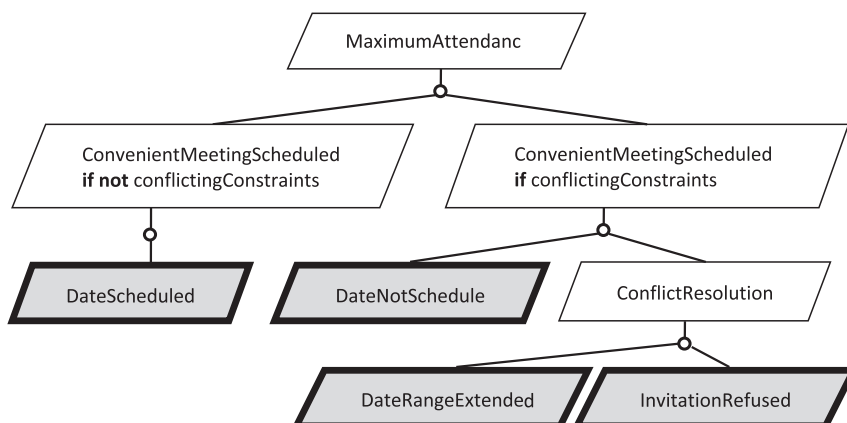


Fig. 12. A fragment of the Meeting Scheduler goal model.

In the first case, the purpose is the definition of the effect (pre and postconditions) of the *operation model* and refining the *object model* (target artifacts) according to the *goal model*, the preliminary definition of the *object model* and the list of operations to realize each leaf goal (source artifacts). The TDCM iteration sequence would be determined by the operations list and the goals that they intend to *operationalize*. For each operation, there should be at least one TDCM test case that builds an initial state based on the associated goal description (CSL allows defining IB states with specific STL statements, instead of asserting the occurrence of events [11]) and tests the occurrence of the operation. The iteration sequence ends once all operations have been considered. We also obtain a resulting test set that validates that each operation correctly *operationalizes* its associated goal in at least one representative case.

In the second case, the purpose is refining and validating the *object model* and the *operation model* (target artifacts) by considering the scenarios defined in the *behavioral model* and the previous versions of the *object model* and the *operation model* (source artifacts). The TDCM iteration sequence would be determined by the scenarios of the *behavioral model*. For each scenario, there should be at least one test case. The iteration sequence ends once all scenarios have been considered. The resulting test set validates that both the *operation model* and the *object model* allow the execution of the scenarios of the *behavioral model*.

It is important to note that, as proposed by GORE, TDCM preserves the user intention expressed in goals to the conceptual schema, because the knowledge added to the schema is driven from failures/errors produced by the execution of test cases (which are derived from goal-based models).

As explained in Section 4.1.2, TDCM can detect conflicts between requirements (which can be graphically represented in the goals model). In general, the use of TDCM may suggest changes (additions, updates) to the goals model.

Techniques for goal risk analysis and prioritization are also proposed in goal-oriented methods. These techniques may be useful in deciding the processing order of test cases while applying TDCM.

6. Related work

TDCM is a TDD-based conceptual modeling method in which a conceptual schema is incrementally defined. TDCM is characterized by continuous validation of the schema while it is being developed. In the following, we first review the related work concerning conceptual schema validation (Section 6.1). After that, we review the origins, requirements and variants of the TDD approach, we summarize the published advantages and drawbacks of TDD, and we give our preliminary evaluation of the extent to which they apply to TDCM (Section 6.2).

6.1. Conceptual schema validation

In the context of Model-Driven Development (MDD) [10,17], there has been some work that relates testing to modeling. [44] provides a thorough survey on the state of the art regarding the many relationships between formal specifications and testing. One such relationship, which has attracted a lot of work, is that specifications can be used to automatically derive test cases of the final system and/or of some intermediate artifact. A pioneer work was the SCENT [45] method in which requirements are first elicited and documented in natural language scenarios. The narrative scenarios are then formalized using statecharts. The statecharts are annotated with information important for testing, and test cases are derived by path traversal of statecharts. A representative set of papers of further work in this direction is [46–54]. In these verification approaches, source models are assumed to be valid, and the artifact under test is the code. In contrast, in the conceptual schema testing environment that supports TDCM, the conceptual schema is the artifact under test.

In TDCM, tests contribute to continuously validate the conceptual schema at each iteration, according to stakeholders' needs and expectations. Moreover, conceptual schema testing does not require either the design or the implementation to exist for the purpose of requirements validation.

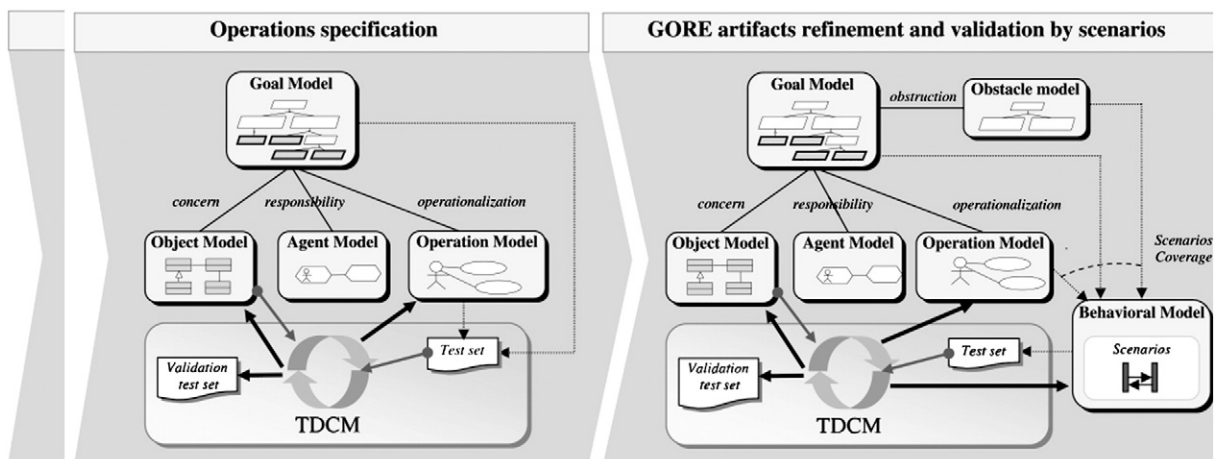


Fig. 13. TDCM integration into a GORE method.

Much effort has also been spent on systematically mapping textual scenarios to conceptual schemas. In [55] we find a work proposing a mapping from scenarios written in natural language to an intermediate conceptual predesign model that can be transformed to a dynamic conceptual model applying mapping heuristics.

Nowadays, requirements validation is still a critical challenge in requirements engineering and, therefore, it is open to new contributions [14,56,57]. In particular, we focus on those requirement validation techniques which are applicable to conceptual schemas (conceptual schema validation).

Some authors [58] state that conceptual schemas specified in logics must be able to answer competency questions, which are requirements formalized in the form of questions. [59,60] propose to translate conceptual schemas into logic to check user-defined properties by reasoning. It is well known that the problem of reasoning with integrity constraints and derivation rules in its full generality is undecidable. Therefore, the available procedures are restricted to certain kinds of constraints and derivation rules or domains, or they may not terminate in some circumstances [61]. In contrast, conceptual schema testing can be applied to schemas that include any kind of OCL constraint or derivation rule and, if there is not an infinite loop in a derivation rule or method, test cases usually terminate in a very short time. The results obtained by testing are not as strong as those obtained by automated reasoning procedures when they are applicable. As has been observed in the general field of software [44], testing and automated reasoning procedures are complementary and can be used in conjunction to enhance validation.

Conceptual schema validation techniques can be used in conjunction with automated procedures aimed at verifying models. There has been a lot of work on automatic verification procedures (a representative set of recent papers is [53,59,60,62–68]). Their general objective is to automatically check that a conceptual schema has a set of properties. The most studied general properties are integrity constraint satisfiability, liveness of an entity or relationship type, non-redundancy of integrity constraints and operation executability.

Domain analysts are able to correctly use concepts of a conceptual schema but have difficulties in abstracting reality in order to represent it through concepts [68]. Paraphrasing techniques generate accurate natural language descriptions of a conceptual schema to be manually validated by domain experts. Some recent work proposes paraphrasing in structured languages [69]. Similar techniques in this respect are the inspections and reviews of requirement specifications [14,70].

Explanation generation is another kind of approach to contribute to the validation of conceptual schemas. explanation generation components can form a question–answer facility about the states of the IB, where explanations are user tailored and may include graphical model views in addition to the textual descriptions [71,72].

Requirement validation is also aimed at checking that the system will correctly prevent invalid states. Constraint acquisition is a recent approach specifically aimed at assisting the elicitation and validation of model constraints [73].

If defined in a formal modeling language, conceptual schemas can be validated by testing techniques that execute them through animation [10,52,74]. The conceptual schema testing environment that supports TDCM belongs to this category of validation techniques. In general, animation facilities allow users to execute operations of the specification with user supplied parameters, thereby calculating the value of the output parameters and the new system state [75].

The idea of animating conceptual schemas for testing purposes dates back to the mid-80s. [76] describes a conceptual language (CPL) and a tool that generates a prototype from a CPL schema, which can be tested. The generated prototype makes it possible to build an IB state, perform consistency checks and ask questions about the contents of the IB. A similar approach was taken in [77] and [78], with the PPP and TROLL light language and environment, respectively.

The most recent work, and the one most closely related to our approach is the USE tool [79–81]. USE makes it possible to define snapshots (instantiations) of structural schemas expressed in a subset of UML/OCL and checking if they are valid instantiations of the schema. [82] is a similar approach focused on behavior, that allows simulation of scenarios (specified as message sequence charts) of use cases modeled in the ADORA language. The TDCM method is supported by a conceptual testing environment (which includes a language and a tool) that allows automated testing in order to execute tests as many times as needed (regression testing). The execution of conceptual schemas in our testing environment extends the USE core to deal with the advanced structural and behavioral schema elements described in Section 2.

6.2. Test-Driven Development

Test-Driven Development (TDD) is a style of development in which (1) an exhaustive suite of test programs is maintained; (2) no code goes into production unless it has associated tests; (3) tests are written first; and (4) the tests determine what code needs to be written [8].

Throughout software engineering history [83], several development process paradigms have been proposed. The traditional waterfall process model was conceived as a sequential development process consisting of five phases: requirement analysis, design, implementation, testing and maintenance. After that, iterative paradigms such as the spiral model or the incremental model were proposed to allow more flexible development processes. In 1998, eXtreme Programming (XP) [6] was introduced as an agile process based on very short development iterations. XP is described in the Agile manifesto [7].

TDD [6,8,84] is a test-first method that emerged in conjunction with the rise of agile paradigms for software development [5]. However, some authors claim that test-first approaches were informally used as early as the 1950s in the NASA's Project Mercury [85]. Most popular development methodologies, like the Unified Process (UP), promote the use of agile practices. TDD is considered to be a key practice of XP. Its particularity is that testing plays a central role in the development process, instead of being a validation phase at the last stages of development. TDD is a test-first approach, in contrast to the traditional test-last approaches.

TDD requires testing environments that provide test automation [5,13] in order to support regression testing. The recent UML Testing Profile (UTP) formally specifies the set of components which may constitute a testing environment [86,87]. Testing environments that allow test automation are known collectively as xUnit frameworks. Such frameworks are based on SUnit [88], originally implemented for Smalltalk. The main ideas of SUnit have been ported to many programming languages and development platforms. Some popular examples [12,89] are JUnit (for testing Java programs), CppUnit (for C++), NUnit (for .NET), XMLUnit, E-Tester (for Eiffel), etc. xUnit testing frameworks support the application of TDD.

Nowadays, an increasing number of organizations are interested in binding requirements and testing more closely together [18,90,91]. Tests may be written in a programming language or in specialized testing languages. Some of these languages are aimed at enhancing understandability of tests for domain people in order to facilitate validation. Fit is a representative framework that promotes the use of business-oriented tables to express storytests [18]. Originally, TDD was focused on the use of unit tests to guide programming. More recently, several researches realize that TDD approaches can contribute to requirement validation by the use of acceptance tests [18,40,92–94]. In the literature, this practice is referred to as Acceptance Test-Driven Development (ATDD), Storytest-Driven Development or Customer Test-Driven Development.

Our proposal is based on the main TDD principles applied to software programming and has the potential of exporting its benefits to conceptual modeling. Several researches have conducted experiments on the effectiveness of TDD [84,95–102]. The first column of Table 5 summarizes seven advantages and five drawbacks indicated by these researches and other published literature. The second column of the table shows our conjectures of the extent to which such advantages and drawbacks may apply to TDCM. Our conjectures are based on our own experience, the large case study referred in Section 4.3 and on the conclusions we drew from the use of TDCM by a group of master students in a recent course project. The table shows that TDCM may “inherit” the seven advantages of TDD and two of its drawbacks, and also that three drawbacks of TDD are not applicable to TDCM. Of course, the conjectures must be confirmed by future rigorous experiments, which we propose to do as further research work.

7. Conclusions

In this paper we have presented the TDCM method, which is an application of TDD to conceptual modeling, and we have shown that it is possible to develop a conceptual schema using it. Using TDCM, conceptual modelers have at any time fully tested schemas.

Table 5

TDD advantages and drawbacks and its potential application to TDCM.

Test-Driven Development (TDD)	Test-Driven Conceptual Modeling (TDCM)
▲ Code quality: Evolving a set of test cases and making them <i>Pass</i> increases code quality (code is added by fixing errors). [5,6,8,40,84,95,100] ▲ Focused objectives: TDD cycle promotes stating and achieving focused objectives instead of worrying about the growing program at any time. Solving small problems, validating them and moving forward is a more predictable way to develop. [5,6,8,40,84,105,106] ▲ Keeping code healthy with refactoring: Refactoring improves the design after adding new code. [5,6,8,40,84,105,106] ▲ Making sure that previous code still works: Automated regression tests help in making sure that code that worked still works after adding new code. [5,6,8,40,84,105,106] ▲ Getting continuous feedback: Code is driven by the feedback provided in test errors and failure information. Therefore, code is evolved as a reaction to continuous feedback. [5,6,8,40,84,105,106] ▲ Workable software at each iteration: After each iteration, you have a code that works according to a set of tests. [5,6,8,40,84] ▲ Programmer confidence: Code is testable as soon as it is written. Together with the previous properties, it increases programmers' confidence. [5,6,8,40,84] ▼ Stakeholders' availability: Stakeholders may not be available during the implementation phase. [106] ▼ Tests as specifications: TDD is “code-centric” and lacks focus on design and analysis. [38,105,106] ▼ Writing test cases is time consuming: Writing test cases consumes time although testing is essential before or after coding. [95,100] ▼ Rewriting tests: When new code is added, previous tests may need to be rewritten. [106] ▼ Tests origin: TDD assumes that developers write tests directly from stakeholders, but does not consider previous analysis or design artifacts. [105,106]	▲ Conceptual schema quality: Evolving a set of test cases and making them <i>Pass</i> increases the semantic quality (correctness and completeness) of the conceptual schema. ▲ Focused objectives: TDCM cycle promotes evolving the conceptual schema incrementally according to focused objectives. Adding knowledge to the schema according to new expectations, validating them and moving forward is a more predictable way to perform conceptual modeling. ▲ Keeping conceptual schema specification healthy: Refactoring improves the way in which the conceptual schema is modeled. ▲ Making sure that already defined knowledge is still valid: After adding new knowledge to the schema, regression testing helps in detecting undesired collateral effects on previously defined knowledge. ▲ Getting continuous feedback: Conceptual modeling is driven by the feedback provided in test errors and failure information. Therefore, new knowledge is added as a reaction to continuous feedback. ▲ Complete and correct schema at each iteration: After each iteration, the result is a correct and complete conceptual schema according to the processed test cases. ▲ Conceptual modeler confidence: The conceptual schema is continuously validated by execution of conceptual test cases. Together with the previous properties, it increases conceptual modelers' confidence. □ Stakeholders' availability: The availability of stakeholders is inherent to conceptual modeling (as a requirements engineering activity). □ Development of the conceptual schema: The aim of TDCM is to define the conceptual schema of an IS, which is a requirements artifact. ▼ Writing test cases is time consuming: Writing test cases for testing the conceptual schema consumes time (during or after developing the schema) although conceptual schema testing increases its quality. ▼ Rewriting tests: Adding new knowledge to the CSUT may necessitate rewriting previous passing test cases. Guidelines are proposed in Section 4.4. to minimize this drawback. □ TDCM is applied in the context of a general method: Test cases may be derived from artifacts provided by the general method that embeds TDCM.

▲ Advantage ▼ Drawback □ Non-applicable.

TDCM fosters the development of correct and complete schemas, which are the fundamental quality properties in conceptual modeling [2,103,104]. We have seen that, using TDCM, a system's conceptual schema is incrementally obtained by performing three kinds of tasks: (1) Write a test the system should pass; (2) Change the schema to pass the test; and (3) Refactor the schema to improve its qualities. We have also presented a set of two guidelines on the use of TDCM. We have dealt with schemas written in UML/OCL, but TDCM can be adapted for the development of schemas in other languages. We have explored the integration of TDCM into four well-known requirements engineering methods: the Unified Process development methodology, the MDD-based approaches, the storytest-driven agile methods and the goal and scenario-oriented methods.

As far as we know, this is the first work that explores the use of TDD in conceptual modeling, and naturally a lot of work remains to be done. We mention three research directions here. The first direction is to further elaborate the integration approach of TDCM into the above mentioned requirements engineering methods, and analyze the results of its use in practice. The second direction aims at experimentally determining to what extent TDCM brings the advantages and drawbacks of TDD to conceptual modeling. This may depend on the requirements engineering method in which TDCM is applied. We have conjectured (in Section 6.2) that TDCM may "inherit" several advantages and drawbacks but they must be confirmed by rigorous development experiments in practical projects. We are confident that TDCM will be very useful in those projects that follow the OMG's Model-Driven Development approach when the transformation from Platform-Independent Models to Platform-Specific Models is fully automatic and correct-by-construction, and we conjecture that TDCM will be useful in the other contexts too, even in projects that develop only the structural schema. The third research direction aims at enlarging the set of guidelines on the use of TDCM, so that they provide useful advice to the conceptual modeler in at least the most common situations.

Acknowledgments

We sincerely thank the anonymous referees for their valuable comments and suggestions, which have improved the paper. We would also like to thank the people of the GMC group for their useful comments on previous drafts of this paper. This work has been partly supported by the Ministerio de Ciencia y Tecnología and FEDER under the project TIN2008-00444, Grupo Consolidado.

References

- [1] A. Olivé, Conceptual Modeling of Information Systems, Springer, Berlin, 2007.
- [2] O.I. Lindland, G. Sindre, A. Solvberg, Understanding quality in conceptual modeling, *IEEE Software* 11 (1994) 42–49.
- [3] A.R. Hevner, S.T. March, J. Park, S. Ram, Design science in information systems research, *Mis Quarterly* (2004) 75–105.
- [4] A. Olivé, Conceptual schema-centric development: a grand challenge for information systems research, 17th Conference on Advanced Information Systems Engineering (CAISE'05), LNCS, Springer, Porto, Portugal, 2005, pp. 1–15.
- [5] D. Janzen, H. Saiedian, Test-driven development: concepts, Taxonomy, and Future Direction, *Computer* 38 (2005) 43–50.
- [6] K. Beck, Test-Driven Development: By Example, Addison-Wesley, 2003.
- [7] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, et al., Manifesto for Agile Software Development, <http://www.agilemanifesto.org/> 2010 accessed July).
- [8] D. Astels, Test Driven Development: A Practical Guide, Prentice Hall, 2003.
- [9] E. Insfrán, V. Pelechano, O. Pastor, Conceptual modeling in the eXtreme, *Information and Software Technology* 44 (2002) 659–669.
- [10] S.J. Mellor, M.J. Balcer, Executable UML, A Foundation for Model-Driven Architecture, Addison-Wesley, 2002.
- [11] A. Tort, A. Olivé, An approach to testing conceptual schemas, *Data & Knowledge Engineering* 69 (2010) 598–618.
- [12] E. Gamma, K. Beck, Available from: JUnit: A Cook's Tour, *Java Report*, 4, 1999, pp. 27–38 <http://junit.sourceforge.net/doc/cookbook/cookbook.htm>.
- [13] British Computer Society, Vocabulary of terms in software testing, British Standards, BS 7925–1, http://www.testingstandards.co.uk/bs_7925-1_online.htm 2010 accessed July.
- [14] A. Van Lamsweerde, Requirements Engineering: From System Goals to UML Models to Software Specifications, Wiley, 2009.
- [15] C. Larman, Applying UML and Patterns, Third edition Prentice Hall, 2005.
- [16] Object Management Group (OMG), MDA Guide, Version 1.0.1, docs/omg/03-06-01, www.omg.org/docs/omg/03-06-01.pdf 2010 accessed July.
- [17] O. Pastor, J.C. Molina, Model-Driven Architecture in Practice, Springer, 2007.
- [18] R. Mugridge, Managing agile project requirements with Storytest-Driven Development, *IEEE Software* 25 (2008) 68–75.
- [19] A. Anaby-Tavor, D. Amid, A. Fisher, A. Bercovici, H. Ossher, R. Bellamy, et al., Insights into enterprise conceptual modeling, *Data & Knowledge Engineering* 69 (2010) 1302–1318.
- [20] Object Management Group (OMG), Object Constraint Language Specification, Version 2.2., formal/2010-02-01, <http://www.omg.org/spec/OCL/2.2/> 2010 accessed July.
- [21] A. Olivé, R. Raventós, Modeling events as entities in object-oriented conceptual modeling languages, *Data & Knowledge Engineering* 58 (2006) 243–262.
- [22] M. Albert, J. Cabot, C. Gómez, V. Pelechano, Generating operation specifications from UML class diagrams: a model transformation approach, *Data & Knowledge Engineering* 70 (2011) 365–389.
- [23] B. Meyer, Object-Oriented Software Construction, Second edition Prentice Hall, 1997.
- [24] G. Booch, J. Rumbaugh, I. Jacobson, The Unified Modeling Language Reference Manual, Second edition Addison-Wesley, 2005.
- [25] B. Meyer, Seven principles of software testing, *IEEE Computer* 41 (2008) 99–101.
- [26] A. Tort, The osCommerce Conceptual Schema Available from: <http://guifre.lsi.upc.edu/OSCommerce.pdf> 2007.
- [27] A. Tort, Testing the osCommerce Conceptual Schema by Using CSTL, Research Report LSI-09-28-R, UPC Available from: <http://hdl.handle.net/2117/6289> 2009.
- [28] T. Mens, T. Tourwé, A survey of software refactoring, *IEEE Transactions on Software Engineering* 30 (2004) 126–139.
- [29] G. Sunyé, D. Pollet, Y. Le Traon, J.M. Jézéquel, Refactoring UML models, 4th International Conference on the Unified Modeling Language (UML 2001), LNCS, vol. 2185, Springer, 2001, pp. 134–148.
- [30] J. Zhang, Y. Lin, J. Gray, Generic and domain-specific model refactoring using a model transformation engine, in: S. Beydeda, M. Book, V. Gruhn (Eds.), Model-Driven Software Development, Springer, 2005, pp. 199–218.
- [31] S.R. Judson, D.L. Carver, R.B. France, A metamodeling approach to model transformation, Conference on Object Oriented Programming Systems Languages and Applications, ACM New York, NY, USA, 2003, pp. 326–327.
- [32] I. Porres, Model refactorings as rule-based update transformations, 6th International Conference on the Unified Modeling Language (UML 2003), LNCS, vol. 2863, Springer, 2003, pp. 159–174.
- [33] A. Correa, C. Werner, Applying refactoring techniques to uml/ocl models, 7th International Conference on the Unified Modeling Language (UML 2004), LNCS, vol. 3273, Springer, 2004, pp. 173–187.

- [34] osTicket, osTicket website, <http://www.osticket.com/> 2011.
- [35] A. Tort, Development of the Conceptual Schema of the osTicket System by Applying TDCM, Research Report UPC Available from: <http://hdl.handle.net/2117/12369> 2011.
- [36] E.J. Chikofsky, J.H. Cross II, Reverse engineering and design recovery: a taxonomy, *IEEE Software* (1990) 13–17.
- [37] E. Kavakli, P. Loucopoulos, Goal modeling in requirements engineering: analysis and critique of current methods, *Information Modeling Methods and Methodologies*, vol. 3273, 2005, pp. 173–187.
- [38] B. Boehm, Get ready for agile methods, with care, *Computer* 35 (2002) 64–69.
- [39] P. Nieto, D. Costal, C. Gómez, Enhancing the semantics of UML association redefinition, *Data & Knowledge Engineering* 70 (2010) 182–207.
- [40] L. Koskela, Test Driven: Practical TDD and Acceptance TDD for Java Developers, Manning Publications, 2007.
- [41] W.N. Robinson, S.D. Pawlowski, V. Volkov, Requirements interaction management, *ACM Computing Surveys (CSUR)* 35 (2003) 132–190.
- [42] C. Rolland, C. Salinesi, Modeling goals and reasoning with them, in: A. Aurum, C. Wohlin (Eds.), *Engineering and Managing Software Requirements*, Springer, 2005, pp. 189–217.
- [43] K. Pohl, *Requirements engineering, Fundamentals, Principles, and Techniques*, Springer, Berlin, 2010.
- [44] R.M. Hierons, K. Bogdanov, J.P. Bowen, R. Cleaveland, J. Derrick, J. Dick, et al., Using formal specifications to support testing, *ACM Computing Surveys*, vol. 41, 2009.
- [45] J. Ryser, M. Glinz, A practical approach to validating and testing software systems using scenarios, *QWE'99*, 3rd International Software Quality Week Europe, Citeseer, 1999.
- [46] S. Pickin, C. Jard, Y. Le Traon, T. Jéron, J.M. Jézéquel, A. Le Guennec, System test synthesis from UML models of distributed software, 22nd International Conference on Formal Techniques for Networked and Distributed Systems (FORTE 2002), LNCS, vol. 2529, Springer, 2002, pp. 97–113.
- [47] A. Gargantini, C. Heitmeyer, Using model checking to generate tests from requirements specifications, *ACM SIGSOFT Software Engineering Notes*, vol. 24, 1999, pp. 146–162.
- [48] K. Nebut, F. Fleurey, Y. Le Traon, J.M. Jézéquel, Automatic test generation: a use case driven approach, *IEEE Transactions on Software Engineering* 32 (2006) 140–155.
- [49] P. Santos Neto, R. Resende, C. Padua, A method for information systems testing automation, The 17th International Conference on Advanced Information Systems Engineering (CAISE'05), LNCS, vol. 3520, Springer, 2005, pp. 504–518.
- [50] A. Hartman, K. Nagin, The AGEDIS tools for model based testing, *ACM SIGSOFT Software Engineering Notes*, vol. 29, 2004, pp. 129–132.
- [51] L. Briand, Y. Labiche, A UML-based approach to system testing, 4th International Conference on the Unified Modeling Language (UML 2001), LNCS, vol. 2185, Springer, 2001, pp. 149–208.
- [52] Y. Zhang, Test-driven modeling for model-driven development, *IEEE Software* 21 (2004) 80–86.
- [53] O. Pilskalns, A. Andrews, A. Knight, S. Ghosh, R. France, Testing UML designs, *Information and Software Technology* 49 (2007) 892–912.
- [54] A.Z. Javed, P.A. Strooper, G.N. Watson, Automated generation of test cases using model-driven architecture, in: 2nd International Workshop on Automation of Software Test (AST 2007), IEEE Computer Society, Washington, DC, USA, 2007.
- [55] G. Fliedl, C. Kop, H.C. Mayr, From textual scenarios to a conceptual schema, *Data & Knowledge Engineering* 55 (2005) 20–37.
- [56] J. Parsons, Effects of local versus global schema diagrams on verification and communication in conceptual data modeling, *Journal of Management Information Systems* 19 (2002) 155–183.
- [57] V. Khatri, I. Vessey, V. Ramesh, P. Clay, S. Park, Understanding conceptual schemas: exploring the role of application and IS domain knowledge, *Information Systems Research* 17 (2006) 81.
- [58] M. Grüninger, M.S. Fox, Methodology for the design and evaluation of ontologies, *Workshop on Basic Ontological Issues in Knowledge Sharing*, Citeseer, 1995.
- [59] A. Queralt, E. Teniente, Decidable reasoning in UML schemas with constraints, The 20th International Conference on Advanced Information Systems Engineering (CAISE'08), LNCS, vol. 5074, Springer, 2008, pp. 281–295.
- [60] A. Queralt, E. Teniente, Reasoning on UML conceptual schemas with operations, The 21st International Conference on Advanced Information Systems Engineering (CAISE'09), LNCS, vol. 5565, Springer, 2009, pp. 47–62.
- [61] A. Queralt, Validation of UML conceptual schemas with OCL constraints and operations, Ph.D. Thesis, Universitat Politècnica de Catalunya, 2009.
- [62] A. Formica, Satisfiability of object-oriented database constraints with set and bag attributes, *Information Systems* 28 (2003) 213–224.
- [63] D. Berardi, D. Calvanese, G. De Giacomo, Reasoning on UML class diagrams, *Artificial Intelligence* 168 (2005) 70–118.
- [64] M. Jarraar, Towards automated reasoning on ORM schemes, 26th International Conference on Conceptual Modeling (ER 2007), LNCS, vol. 4801, Springer, 2007, pp. 181–197.
- [65] J. Cabot, R. Clarisó, D. Riera, Verification of UML/OCL class diagrams using constraint programming, *MoDeVva 2008. ICST Workshop*, Citeseer, 2008, pp. 73–80.
- [66] A. Kalyanpur, B. Parsia, E. Sirin, J. Hendler, Debugging unsatisfiable classes in OWL ontologies, *Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 3, 2005, pp. 268–293.
- [67] M. Glinz, A lightweight approach to consistency of scenarios and class models, 4th International Conference on Requirements Engineering (ICRE 2000), 2000, pp. 49–58.
- [68] C. Rolland, C. Proix, A natural language approach for requirements engineering, The 5th International Conference on Advanced Information Systems Engineering (CAISE'93), LNCS, vol. 593, Springer, 1992, pp. 257–277.
- [69] J. Cabot, R. Pau, R. Raventós, From UML/OCL to SBVR specifications: a challenging transformation, *Information Systems* 35 (2010) 417–440.
- [70] E.P. Doolan, Experience with Fagan's inspection method, *Software: Practice and Experience* 22 (1992) 173–182.
- [71] J.A. Gulla, G. Willumsen, Using explanations to improve the validation of executable models, 5th International Conference on Advanced Information Systems Engineering (CAISE 1993), LNCS, vol. 685, Springer, 1993, pp. 118–142.
- [72] A. Olivé, M.R. Sancho, Validating conceptual specifications through model execution, *Information Systems* 21 (1996) 167–186.
- [73] S. Hartmann, S. Link, T. Trinh, Constraint acquisition for entity-relationship models, *Data & Knowledge Engineering* 68 (2009) 1128–1155.
- [74] J.S. Ostroff, F.A. Torshizi, Testable requirements and specifications, 3rd International Conference on Tests And Proofs (TAP 2007), LNCS, vol. 4454, Springer, 2007, p. 17.
- [75] J. Bicarregui, J. Dick, B. Matthews, E. Woods, Making the most of formal specification through animation, *Testing and Proof*, Science of Computer Programming 29 (1997) 53–78.
- [76] F. Dignum, T. Kemme, W. Kreutzen, H. Weigand, R.P. van de Riet, Constraint modelling using a conceptual prototyping language, *Data & Knowledge Engineering* 2 (1987) 213–254.
- [77] O.I. Lindland, J. Krogstie, Validating conceptual models by transformational prototyping, 5th International Conference on Advanced Information Systems Engineering (CAISE 1993), LNCS, vol. 685, Springer, 1993, pp. 165–183.
- [78] M. Gogolla, S. Conrad, G. Denker, R. Herzig, N. Vlachantonis, A development environment for an object specification language, *IEEE Transactions on Knowledge and Data Engineering* 7 (1995) 505–508.
- [79] M. Gogolla, F. Büttner, M. Richters, USE: a UML-based specification environment for validating UML and OCL, *Science of Computer Programming* 69 (2007) 27–34.
- [80] M. Gogolla, J. Bohling, M. Richters, Validating UML and OCL Models in USE by automatic snapshot generation, *Software & Systems Modeling* 4 (2005) 386–398.
- [81] M. Richters, M. Gogolla, Validating UML Models and OCL Constraints, 3rd International Conference on the Unified Modeling Language (UML 2000), LNCS, Springer, York, UK, 2000, pp. 265–277.
- [82] C. Seybold, S. Meier, M. Glinz, Scenario-driven modeling and validation of requirements models, *International Workshop on Scenarios and State Machines: Models, Algorithms, and Tools*, ACM, 2006.
- [83] I. Sommerville, *Software Engineering*, 9th edition Addison-Wesley, 2010.
- [84] D. Janzen, H. Saiedian, Does test-driven development really improve software design quality? *IEEE Software* 25 (2008) 77–84.
- [85] C. Larman, V.R. Basili, Iterative and incremental developments. a brief history, *Computer* 36 (2003) 47–56.

- [86] P. Baker, Z.R. Dai, J. Grabowski, Ø. Haugen, I. Schieferdecker, C. Williams, *Model-Driven Testing: Using the UML Testing Profile*, Springer, 2008.
- [87] Object Management Group (OMG), *UML Testing Profile, Version 1.0, formal/05-07-07*, 2010 accessed July.
- [88] K. Beck, *Simple Smalltalk Testing: With Patterns*, Research Report Smalltalk 4, SmalltalkAvailable from: <http://www.xprogramming.com/ftp/TestingFramework/Testfram.rtf> 1994.
- [89] J.S. Ostroff, R.F. Paige, D. Makalsky, P.J. Brooke, E-tester: a contract-aware and agent-based unit testing framework for Eiffel, *Journal of Object Technology* 4 (2005) 97–114.
- [90] E.J. Uusitalo, M. Komssi, M. Kauppinen, A.M. Davis, Linking requirements and testing in practice, 16th IEEE International Requirements Engineering (RE'08), 2008, pp. 265–270.
- [91] R.C. Martin, G. Melnik, O.M. Inc, Tests and requirements, requirements and tests: a Möbius strip, *IEEE Software* 25 (2008) 54–59.
- [92] G. Melnik, F. Maurer, M. Chiasson, Executable acceptance tests for communicating business requirements: customer perspective, *Agile 2006 Conference*, IEEE Computer Press, 2006, pp. 35–46.
- [93] G. Melnik, F. Maurer, Multiple perspectives on executable acceptance test-driven development, *International Conference on Agile Processes in Software Engineering and Extreme Programming (XP 2007)*, LNCS, vol. 4536, Springer, 2007, pp. 245–249.
- [94] J.P. Sauv  , A. Neto, O. Lopes, Teaching software development with ATDD and EasyAccept, *ACM SIGCSE Bulletin* 40 (2008) 542–546.
- [95] E.M. Maximilien, L. Williams, Assessing test-driven development at IBM, 25th International Conference on Software Engineering (ICSE'03), IEEE Computer Society, Washington DC, USA, 2003, pp. 564–569.
- [96] A. Geras, M. Smith, J. Miller, A prototype empirical evaluation of test driven development, 10th International Symposium on Software Metrics (METRICS'04), 2004, pp. 405–416.
- [97] K.M. Lui, K.C. Chan, Test Driven Development and Software Process Improvement in China, in: 5th International Conference on Extreme Programming and Agile Processes in Software Engineering (XP 2005), Eckstein and Baumeister, Springer, 2004, pp. 219–222.
- [98] L. Madeyski, L. Sza  , The Impact of Test-Driven Development on Software Development Productivity-An Empirical Study, in: P. Abrahamsson, N. Baddoo, T. Margaria, R. Messnarz (Eds.), *Software Process Improvement*, LNCS, Springer, 2007, pp. 200–211.
- [99] M. Muller, O. Hagner, Experiment about test-first programming, *IEE Proceedings Software* 149 (2002) 131–136.
- [100] B. George, L. Williams, An initial investigation of test driven development in industry, 2003 ACM symposium on applied computing, ACM Press, New York, NY, USA, 2003, pp. 1135–1139.
- [101] T. Bhat, N. Nagappan, Evaluating the efficacy of test-driven development: industrial case studies, 2006 ACM/IEEE International Symposium on Empirical Software Engineering, ACM Press, New York, NY, USA, 2006, pp. 356–363.
- [102] G. Canfora, A. Cimitile, F. Garcia, M. Piattini, C.A. Visaggio, Evaluating advantages of test driven development: a controlled experiment with professionals, 2006 ACM/IEEE International symposium on empirical software engineering, ACM Press, New York, 2006, pp. 364–371.
- [103] D.L. Moody, Theoretical and practical issues in evaluating the quality of conceptual models: current state and future directions, *Data & Knowledge Engineering* 55 (2005) 243–276.
- [104] J. Recker, M. Rosemann, The measurement of perceived ontological deficiencies of conceptual modeling grammars, *Data & Knowledge Engineering* 69 (2010) 516–532.
- [105] M. Stephens, D. Rosenberg, *Design Driven Testing*, Apress, New York, 2010.
- [106] M. Stephens, D. Rosenberg, *Extreme Programming Refactored: The Case Against XP*, Apress, New York, 2003.
- [107] B. Nuseibeh, S. Easterbrook, The process of inconsistency management: a framework for understanding, *First International Workshop on the Requirements Engineering Process (REP'99)*, IEEE, 1999, pp. 364–368.



Albert Tort is currently a researcher at the Services and Information Systems Engineering Department of the Universitat Polit  cnica de Catalunya in Barcelona. He is a member of the Information Modeling and Processing (MPI) research group. He obtained his Computer Science degree in 2008 from the Universitat Polit  cnica de Catalunya and he received his M.Sc. in Computing in 2009. He is working towards his Ph.D. degree. His main research interests are conceptual modeling of information systems and requirements engineering.



Antoni Oliv   is a professor of information systems at the Universitat Polit  cnica de Catalunya in Barcelona. He has worked in this field for over 20 years, mainly in the university and research environments. His main interests have been, and are, conceptual modeling, requirements engineering, information systems design and databases. He has taught extensively on these topics. He has also conducted research on these topics, which has been published in international journals and conferences. He is a member of IFIP WG8.1 (design and evaluation of information systems) where he served as chairman during 1989–1994.



Maria-Ribera Sancho is a tenured professor at the Universitat Polit  cnica de Catalunya (Barcelona Tech). She has been active in the software engineering field for over 20 years. She has served as Dean of the Barcelona School of Informatics (2004–2010). Her main research interests are conceptual modeling, information systems design and software engineering. Now her interests include service science (SSME). She is the author of different research articles and papers for International Congresses and Journals. She is the Spanish representative at the IFIP TC8 (Information Systems).